Second-order generalised algebraic theories, by examples (under constructon) vizsgára szükséges részek

Kaposi Ambrus jegyzete alapján

May 31, 2026

In these notes, we study non-substructural programming languages at a high level of abstraction: a language is a second-order generalised algebraic theory (SOGAT). The syntax of such a language is its initial model, in which there are only well-typed (intrinsic) terms quotiented by conversion, every operation is automatically a congruence with respect to conversion and every operation is stable under substitution. These notes are pedagogical excerpts of the papers [KX24, BKS23]. A more categorical treatment can be found in the upcoming PhD thesis of Rafaël Bocquet.

Related ideas are higher-order abstract syntax [Hof99], logical frameworks [HHP93], two-level type theories [ACKS23], synthethic Tait computability [Ste22]. We will rely on being able to work informally in type theory (informal Agda, Coq, Idris or Lean).

These notes can be formalised in the following metatheories: observational type theory, extensional type theory with quotient inductive-inductive types and propositional extensionality, homotopy type theory, constructive set theory.

These notes were originally written for the International School on Logical Frameworks and Proof Systems Interoperability (LFPSI) which was in Orsay between 8–11 September 2025.

Thanks to Botond Mészáros and Vojtěch Štěpančík for fixing typos.

Sections compulsory for the type systems course at ELTE: 1, 2.3, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7.2, 3.8.

"In these notes, we study non-substructural programming languages at a high level of abstraction"

Imagine you want to describe what a recipe **is**, without worrying about whether it's written on paper or a whiteboard or in a book. You want to talk about the **essence** of a recipe — the ingredients and steps — not the handwriting or the font. That's what "high level of abstraction" means. We want to describe programming languages by their **essence** — what programs mean and how they run — not the fiddly details of how they're written down.

"Non-substructural" just means we're looking at normal, everyday programming languages where you can use a variable as many times as you want. We're not looking at exotic languages where, say, you must use each variable exactly once.

"A language is a second-order generalised algebraic theory (SOGAT)"

This is the big claim of the whole course. A SOGAT is a precise mathematical recipe for describing a programming language at that high level of abstraction. The word "second-order" refers to the fact that some operations in the language take **functions as input** — like lam, which takes a function Tm A → Tm B and packages it into a term. That's what makes it "second-order" rather than just a plain algebraic theory.

"The syntax of such a language is its initial model"

A **model** of a language is any mathematical structure that satisfies all the rules of the language. There can be many models — the "standard" one where programs actually compute things, weird ones, big ones, small ones.

The **initial model** is the smallest, most canonical one — the one that contains **exactly** the programs you can write, nothing more and nothing less. This is what we call the **syntax**. It's "initial" in the sense that there is a unique map from it into any other model.

"In which there are only well-typed (intrinsic) terms quotiented by conversion"

Two things here:

Well-typed intrinsic terms — as we discussed, ill-typed programs simply don't exist in the syntax. You can't even write isZero true — it's not a thing.

Quotiented by conversion — two programs that compute to the same result are treated as literally the same program. num 0 + num 1 and num 1 are not just *equivalent*, they are **identical** in the syntax.

"Every operation is automatically a congruence with respect to conversion and every operation is stable under substitution"

Congruence means: if two subterms are equal, the bigger terms built from them are also equal. If a = a', then num a + num 1 = num a' + num 1 automatically. You don't have to prove this separately for each operation — it comes for free from how the syntax is built.

Stable under substitution means: if you replace a variable with a term, the result is still a valid term of the right type. Again, this comes for free — you don't need a separate "substitution lemma".

These are two of the most painful things to prove when you use a more concrete representation of syntax (like named variables or de Bruijn indices). The whole point of using SOGATs is that you get them for free.

"These notes are pedagogical excerpts of the papers [KX24, BKS23]"

These notes are a simplified, teaching-friendly version of research papers written by Kaposi and collaborators. The full mathematical machinery is in those papers — the notes focus on building intuition through examples.

"We will rely on being able to work informally in type theory (informal Agda, Coq, Idris or Lean)"

The notes are written in a style that looks like code in a proof assistant — Agda, Coq, etc. You don't need to know any of them formally, but you should be comfortable reading things like Tm : Ty → Set and lam : (Tm A → Tm B) → Tm (A ⇒ B) as if they were type signatures.

"These notes can be formalised in the following metatheories..."

The notes work "informally" — they're written in a style that *could* be made fully rigorous in several different formal systems (extensional type theory, homotopy type theory, etc.). The specific choice doesn't matter much for understanding the ideas.

Tartalomjegyzék

1 Levels of abstraction 4

2.3 Razor 7

3 Derivability in SOGATs 15

3.1 STLC (Simply Typed Lambda Calculus) 16

3.2 System T 25

3.3 Finite types 29

3.4 Generic programming 35

3.5 Inductive types 39

3.6 Coinductive types 44

3.7.2 System F 48

3.8 PCF, Recursive types, Untyped lambda calculus 53

1 Levels of abstraction

A language can be described in different ways ranging from concrete to abstract, see Figure 1. In this section, we explain the different levels briefly via a simple expression language (Razor, see Subsection 2.3). The following example program can be written in Razor.

```
if isZero (num 0 + num 1) then false else isZero (num 0)
```

A Razor program is either a numeric or a boolean expression. Numbers can be formed using `num i` where `i` is a natural number. Booleans are `true` or `false`. We have the usual if-then-else operator, addition and an isZero operator which says whether a number is 0. The above program evaluates (runs) in the following steps (each new line is a step).

```
if isZero (num 0 + num 1) then false else isZero (num 0)
if isZero (num 1) then false else isZero (num 0)
if false then false else isZero (num 0)
isZero (num 0)
true
```

Figure 2 gives complete descriptions of Razor at levels of abstraction (1)–(5):

(1) As a first approximation, a program is a string, that is, a sequence of (ASCII) characters. This is how we write programs on a computer. Any string is a program. Many strings do not correspond to meaningful programs in our language such as `num 3 - num 2` as we don't have subtraction. Also, there are different strings which represent the same program. For example, `isZero  (num 1)` and `isZero (num 1)` are different as strings but should be the same programs as the extra spaces after `isZero` shouldn't matter. Instead of describing which strings are meaningful programs and defining an equivalence relation for identifying strings that represent the same program, we will describe programs using a more abstract structure.

(2) The more abstract structure is list of lexical elements. Now we have much fewer programs and `num 3 - num 2` is not a program anymore because there is no lexical element for `-`. Any two programs given as strings which differ only in the number of spaces will end up as the same program at this level: `isZero  (num 1)` and `isZero (num 1)` are both given by the sequence [`isZero`, `(`, `num`, `1`, `)`]. However, we still have meaningless programs, e.g. [`(`, `true`] (there is no closing parenthesis) or [`num`, `1`, `+`] (`+` needs two arguments), and so on. Also, there are programs which could be identified, e.g. [`(`, `true`, `)`] and [`true`] (the parentheses are redundant in the former). Again, to solve these issues, we move to a higher-level representation of programs. Note that there are standard ways to navigate between levels (1) and (2): (2) to (1) is printing. (1) to (2) is performed by a lexical analyser (lexer) which turns a string into a sequence of lexical elements or returns an error.

(3) Abstract syntax tree descriptions are usually given by BNF grammars. At this level, we only have well-parenthesised expressions and each operator receives the correct number of arguments. Programs are now trees which have `true`, `false` or `num i` at their leaves and they can have ternary branching with `ite` at the branching node, unary branching with `isZero` at the node or binary branching with `+` at the node.

(4) In well-typed (intrinsic) syntax trees, the types of the arguments are restricted to the correct ones.

(5) In well-typed quotiented syntax, two programs which have the same result are identified.

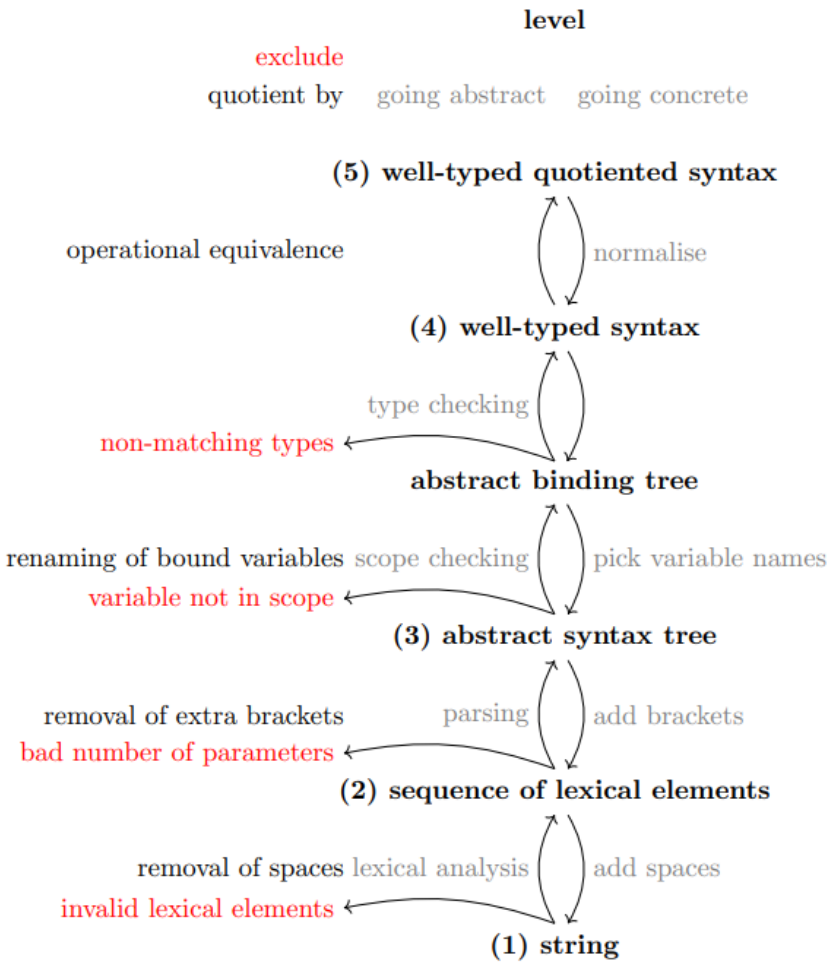


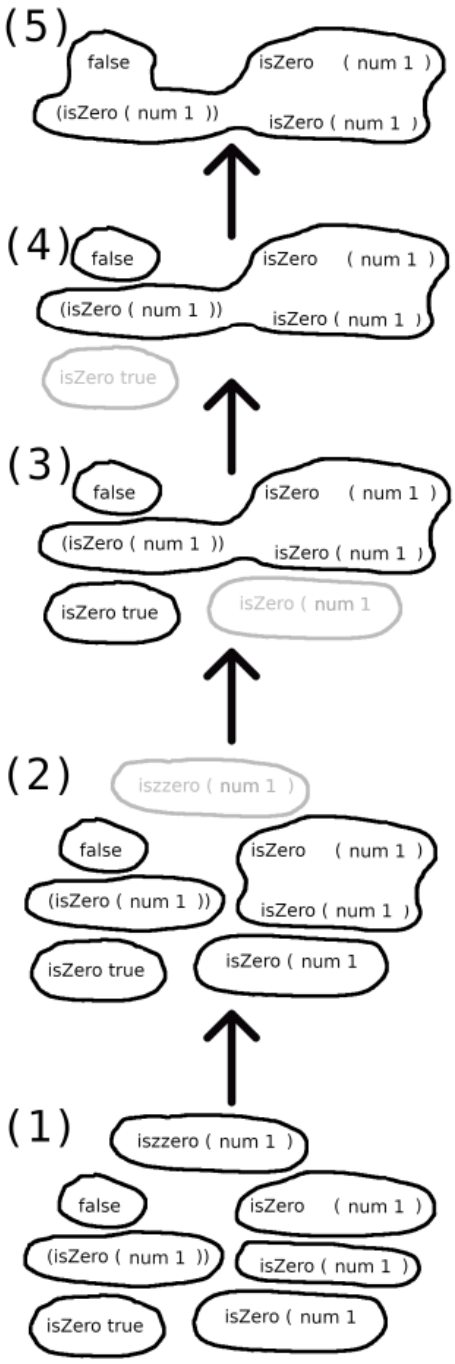
Figure 1: Different levels of abstraction when defining a programming language and transformations between levels. At more abstract levels, certain programs are excluded and others identified. Abstract binding trees are sometimes called well-scoped syntax trees.

Abstract binding trees are not relevant for Razor as there are no variables.

The *extrinsic* approach moves to more abstract levels by defining relations which select the well-formed (well-scoped, well-typed, etc) expressions in the concrete representation. In contrast, we use the *intrinsic* approach where in the more abstract representation the non-well-formed expressions are not even expressible. At level (5), there is no need to prove that conversion preserves typing ("preservation" from "progress and preservation") because the equations are already expressed in a typed way. Certain properties of the language cannot be expressed at the abstract levels. For example it is not possible to count the number of brackets in a program at level (3). At level (5), it is not possible to reason about program efficiency because all convertible programs are in the same equivalence class: we cannot write a function which distinguishes convertible programs. We view the lower levels as important parts of a programming language, but we see them as intermediate technical steps to reach the most abstract level which describes the essence of the language.

In these notes, we will use the most abstract level to describe languages.

There are several features of programming languages that we do not know how to define intrinsically, and these notes only cover a very small fragment of the theory of programming languages. We direct the curious reader to standard textbooks [Pie02, PdAC+23, Har16, WKS22, Csö12, DL11].



(5) well-typed quotiented syntax

```
Ty        :  Set
Tm        :  Ty → Set
Bool      :  Ty
Nat       :  Ty
true      :  Tm Bool
false     :  Tm Bool
ite       :  Tm Bool → Tm A → Tm A → Tm A
num       :  ℕ → Tm Nat
isZero    :  Tm Nat → Tm Bool
_+_       :  Tm Nat → Tm Nat → Tm Nat
iteβ1     :  ite true u v = u
iteβ2     :  ite false u v = v
isZeroβ1  :  isZero (num 0) = true
isZeroβ2  :  isZero (num (1+n)) = false
+β        :  num m + num n = num (m + n)
```

(4) well-typed syntax

```
Ty        :  Set
Tm        :  Ty → Set
Bool      :  Ty
Nat       :  Ty
true      :  Tm Bool
false     :  Tm Bool
ite       :  Tm Bool → Tm A → Tm A → Tm A
num       :  ℕ → Tm Nat
isZero    :  Tm Nat → Tm Bool
_+_       :  Tm Nat → Tm Nat → Tm Nat
```

(3) abstract syntax tree

```
Tm        :  Set
true      :  Tm
false     :  Tm
ite       :  Tm → Tm → Tm → Tm
num       :  ℕ → Tm
isZero    :  Tm → Tm
_+_       :  Tm → Tm → Tm
```

(2) list of the following lexical elements:

`(, ), true, false, if, then, else, num, isZero, +, 0, 1, 2, 3, ...`

(1) any string

Figure 2: Left: example Razor programs at different levels of abstraction. Each bubble represents a separate program. Right: description of the Razor expression language at levels (1)–(5).

2.3 Razor

The following is a simple expression language which is not algebraic, but *generalised* algebraic. There are two sorts, and the second one is indexed over the first one.[1] We call the language Razor following [Hut23].

Definition 14 (Razor)**.** *A model comprises the following components:*

$$
\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
\mathsf{Bool} & : \mathsf{Ty} \\
\mathsf{Nat} & : \mathsf{Ty} \\
\mathsf{true} & : \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{false} & : \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{ite} & : \mathsf{Tm}\,\mathsf{Bool} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A \\
\mathsf{num} & : \mathbb{N} \to \mathsf{Tm}\,\mathsf{Nat} \\
- + - & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Nat} \\
\mathsf{isZero} & : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,\mathsf{Bool} \\
\mathsf{ite}\beta_1 & : \mathsf{ite}\,\mathsf{true}\,u\,v = u \\
\mathsf{ite}\beta_2 & : \mathsf{ite}\,\mathsf{false}\,u\,v = v \\
+\beta & : \mathsf{num}\,m + \mathsf{num}\,n = \mathsf{num}\,(m+n) \\
\mathsf{isZero}\beta_1 & : \mathsf{isZero}\,(\mathsf{num}\,0) = \mathsf{true} \\
\mathsf{isZero}\beta_2 & : \mathsf{isZero}\,(\mathsf{num}\,(1+n)) = \mathsf{false}
\end{array}
$$

We call elements of Ty types and elements of $\mathsf{Tm}\,A$ terms of type A. There is no single set of terms, instead for each type, there is a separate set of terms of that type. Each operation is *typed*: their input types and output type is restricted. The operation ite has one implicit type argument A and its first explicit argument needs to have type Bool, while the second and third explicit arguments need to have (the same) type A.

The derivable operations can be seen as programs that can be defined in this language, e.g. $\mathsf{not}\,(b : \mathsf{Tm}\,\mathsf{Bool}) : \mathsf{Tm}\,\mathsf{Bool} := \mathsf{ite}\,b\,\mathsf{false}\,\mathsf{true}$. The equations explain how to run the programs, e.g. $\mathsf{not}\,\mathsf{true} = \mathsf{ite}\,\mathsf{true}\,\mathsf{false}\,\mathsf{true} \overset{\mathsf{ite}\beta_1}{=} \mathsf{false}$. Another example:

$$
\begin{array}{ll}
(\mathsf{num}\,1 + \mathsf{num}\,2) + (\mathsf{num}\,3 + \mathsf{num}\,4) & =(+\beta) \\
(\mathsf{num}\,(1+2)) + (\mathsf{num}\,3 + \mathsf{num}\,4) & = \\
\mathsf{num}\,3 + (\mathsf{num}\,3 + \mathsf{num}\,4) & =(+\beta) \\
\mathsf{num}\,3 + \mathsf{num}\,(3+4) & = \\
\mathsf{num}\,3 + \mathsf{num}\,7 & =(+\beta) \\
\mathsf{num}\,(3+7) & = \\
\mathsf{num}\,10 &
\end{array}
$$

Note that in the expression $\mathsf{num}\,(1+2)$, the $+$ refers to the metatheoretic addition, while in $\mathsf{num}\,1 + \mathsf{num}\,2$ the $+$ refers to object theoretic addition.

The standard (metacircular, set) model of Razor is where types are sets (metatheoretic types) and terms

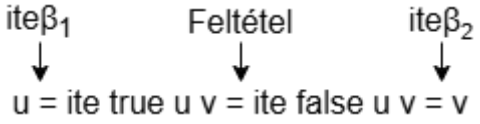
[1] Certain such GATs can be reduced to ATs by replacing the Ty-indexing with a function from Tm to Ty. For Razor, this is not the case because after such a reduction, the ite operation becomes partial. The resulting theory is called an essentially algebraic theory (EAT). EATs are another extension of algebraic theories. GATs and EATs are equivalent in the sense that for each GAT, there is an EAT with an equivalent category of models, and for each EAT, there is a GAT with an equivalent category of models.

are elements of the sets (metatheoretic terms):

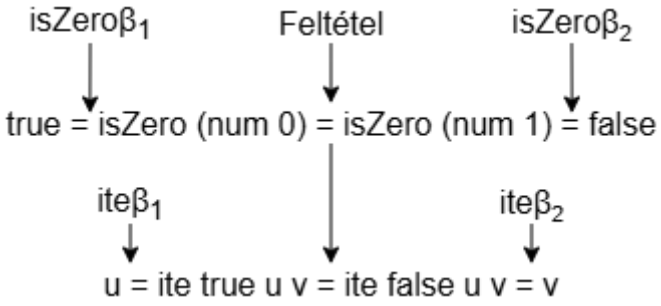
$$
\begin{array}{ll}
\mathsf{Ty} & := \mathsf{Set} \\
\mathsf{Tm}\, A & := A \\
\mathsf{Bool} & := \{\mathsf{tt}, \mathsf{ff}\} \\
\mathsf{Nat} & : \mathbb{N} \\
\mathsf{true} & := \mathsf{tt} \\
\mathsf{false} & := \mathsf{ff} \\
\mathsf{ite}\, b\, t\, f & := \mathsf{match}\, b\, \{\mathsf{tt} \mapsto t; \mathsf{ff} \mapsto f\} \\
\mathsf{num}\, n & := n \\
u + v & := u + v \\
\mathsf{isZero}\, u & := \mathsf{match}\, u\, \{0 \mapsto \mathsf{tt}; (1+n) \mapsto \mathsf{ff}\}
\end{array}
$$

We do not write metatheoretic universe levels, hence informally Set : Set, but formally types in the standard model are a large metatheoretic type (they are in Set_1). In the definition of ite and isZero we used pattern matching on elements of metatheoretic booleans {tt, ff}. In the standard model, all equations hold by reflexivity, as they hold definitionally in the metatheory.

Exercise 15. *If in any model* true = false *then for any* A *and* $u, v : \mathsf{Tm}\, A$*, we have* $u = v$.



Exercise 16. *If in any model* num 0 = num 1 *then for any* A *and* $u, v : \mathsf{Tm}\, A$*, we have* $u = v$.



Exercise 17. *There is a model where* num 1 = num 2*, but* true ≠ false.

A Bool maradhat ugyanúgy kételemű, (igaz, hamis) viszont Ty Nat-nál a természetes számok halmaza helyett használjuk korlátolt számok egy véges halmazát, egy olyan halmazt, aminek csak két eleme van, a 0 és az 1: {0, 1}. A 0 a 0-hoz mappelődik, az 1 és minden nála nagyobb szám pedig az 1-hez, tehát talán jobb, ha így jelöljük: {0, 1+}. Ahhoz, hogy ez egy valid modell legyen, a nyelv műveleteinek (isZero, ite, +) konzisztensen definiálhatónak kell lennie ezek fölött a halmazok fölött és meg kell felelniük a számítási szabályoknak (β-szabályok) is:

- **isZero**: isZero (0) = true, isZero (1+) = false (megfelel az isZero (1 + n) = false-nak).
- **ite**: ezt nem nagyon érinti, marad az ite true u v = u, ite false u v = v
- **+**: 0 + x = x, 1+ + x = 1+ (megfelel a num (n) + num (m) = num (n + m)-nek)

Mivel ebben a modellben a num 1 és a num 2 is (1+)-ra mappelődik, így num 1 = num 2 teljesül. A true és false viszont nem kell, hogy egyenlő legyen, tehát adott természetes számok „egymásba omlasztása / ömlesztése" nem omlasztja / ömleszti egymásba automatikusan a bool értékeket is.

Exercise 18. *There is a model where* Tm Bool *has three elements (nonstandard model).*

A Tm Bool definiálható úgy is, hogy három értéket tartalmazzon, pl.: {true, false, γ}. A Tm Nat ettől még maradhat a természetes számok halmaza: {0, 1, 2, 3, 4, 5, 6, 7, ...}. Meg kell nézni, hogy a műveletekkel mi a helyzet:

- **isZero**: 0-ra true-t ad vissza, más értékre false-t mint eddig, a γ-t soha nem adja eredményül, de ez nem baj, mivel nem is elvárás.
- **+**: csak számokkal dolgozik, nem érintik a bool értékek
- **ite**: az ite true u v = u, és ite false u v = v továbbra is fennáll, emellett megadható az új értékhez is egy viselkedés, pl. az, hogy ugyanúgy az első értéket adja vissza: ite γ u v = u.

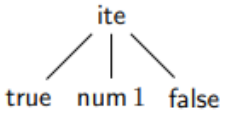
Tehát minden elvárásnak megfelel ez a modell, ezért validnak számít.

Exercise 19. *Prove or disprove the following statements.*

- *There is a model where* true ≠ false *and for every* t *and* u, ite $t\,u\,u = u$.
- *In every model for every* t *and* u, ite $t\,u\,u = u$.
- *There is a model in which* isZero (num 0) = false.
- *In every model* isZero (num 3) = isZero (num 5).
- *There is no model in which* Tm Bool = $\mathbb{N}$.

- **Igaz**: a nyelv standard modelljét használva, ahol a bool érték true és false lehet csak, megadható, hogy ite true u u = u, valamint ite false u u = u, így teljesül.
- **Hamis**: a 18-as feladatban látott Tm Bool = {true, false, γ} esetén pl. megszabhatjuk az ite működését ilyenre: ite γ 0 0 = 1. Így, ha t = γ és u = 0, akkor ite γ 0 0 = 1-et ad vissza, ami viszont nem 0, pedig ezt várná az állítás.
- **Igaz**: Használhatunk egy triviális modellt, ahol minden típus pontosan egy elemet tartalmaz, legyen ez az elem a *. Ekkor true = *, false = * és isZero (num 0) = *, tehát így fennáll az isZero (num 0) = false is.
- **Igaz**: Minden modellnek be kell tartania a számítási szabályokat, (β-szabályok) és mivel isZero (num (n +1)) = false és 3 = 2 + 1, valamint 5 = 4 + 1, így mindkettőre ugyanazt (false-t) ad vissza.
- **Hamis**: Lehet ilyen modellt csinálni. Pl. a true megfelel a 0-nak, a false pedig az 1-nek. Az ite(0, u, v) = u és ite(1, u, v) = v szabályok továbbra is fennállnak majd, minden 1-nél nagyobb számra pedig megszabhatjuk, hogy mit csináljon az ite, pl. adjon mindig vissza 0-t.

Notation 25 (Derivation rules). *Elements of the syntax can be depicted by syntax trees, but then the well-typedness of the tree is not immediately visible, it is not enforced by the drawing that the tree does not make sense. Instead, we first describe the notion of model using derivation rule notation which means that we uncurry all the operators, give names to the arguments and in each operator, we replace the remaining* $\to$ *with a horizontal line:*



$$\frac{}{\mathsf{Ty} : \mathsf{Set}} \qquad \frac{A : \mathsf{Ty}}{\mathsf{Tm}\, A : \mathsf{Set}} \qquad \frac{}{\mathsf{Bool} : \mathsf{Ty}} \qquad \frac{}{\mathsf{Nat} : \mathsf{Ty}} \qquad \frac{}{\mathsf{true} : \mathsf{Tm}\,\mathsf{Bool}} \qquad \frac{}{\mathsf{false} : \mathsf{Tm}\,\mathsf{Bool}}$$

$$\frac{b : \mathsf{Tm}\,\mathsf{Bool} \qquad t : \mathsf{Tm}\, A \qquad f : \mathsf{Tm}\, A}{\mathsf{ite}\, b\, t\, f : \mathsf{Tm}\, A} \qquad \frac{n : \mathbb{N}}{\mathsf{num}\, n : \mathsf{Tm}\,\mathsf{Nat}} \qquad \frac{u : \mathsf{Tm}\,\mathsf{Nat} \qquad v : \mathsf{Tm}\,\mathsf{Nat}}{u + v : \mathsf{Tm}\,\mathsf{Nat}} \qquad \frac{u : \mathsf{Tm}\,\mathsf{Nat}}{\mathsf{isZero}\, u : \mathsf{Tm}\,\mathsf{Bool}}$$

$$\frac{}{\mathsf{ite}\beta_1 : \mathsf{ite}\,\mathsf{true}\, u\, v = u} \qquad \frac{}{\mathsf{ite}\beta_2 : \mathsf{ite}\,\mathsf{false}\, u\, v = v} \qquad \frac{}{+\beta : \mathsf{num}\, m + \mathsf{num}\, n = \mathsf{num}\,(m + n)}$$

$$\frac{}{\mathsf{isZero}\beta_1 : \mathsf{isZero}\,(\mathsf{num}\, 0) = \mathsf{true}} \qquad \frac{}{\mathsf{isZero}\beta_2 : \mathsf{isZero}\,(\mathsf{num}\,(1 + n)) = \mathsf{false}}$$

Just as in the algebraic notation we have implicit arguments, for example $A : \mathsf{Ty}$ *is not written above the line for* ite*, but we assume it is (implicitly) there.*

Notation 26 (Derivation trees). *Derivation trees are like upside-down syntax trees where at the nodes the full term and its set are both written; the children are subterms just as before. Example:*

$$\frac{\dfrac{\dfrac{}{\mathsf{num}\, 1 : \mathsf{Tm}\,\mathsf{Nat}}}{\mathsf{isZero}\,(\mathsf{num}\, 1) : \mathsf{Tm}\,\mathsf{Bool}} \qquad \frac{}{\mathsf{true} : \mathsf{Tm}\,\mathsf{Bool}} \qquad \frac{}{\mathsf{false} : \mathsf{Tm}\,\mathsf{Bool}}}{\mathsf{ite}\,(\mathsf{isZero}\,(\mathsf{num}\, 1))\,\mathsf{true}\,\mathsf{false} : \mathsf{Tm}\,\mathsf{Bool}}$$

Each horizontal line in the derivation tree is a special case of one of the derivation rules.

It is clear that the following tree cannot be finished:

$$\frac{\dfrac{}{\mathsf{true} : \mathsf{Tm}\,\mathsf{Bool}} \qquad \dfrac{}{\mathsf{num}\, 1 : \mathsf{Tm}\,\mathsf{Nat}} \qquad \dfrac{????}{\mathsf{false} : \mathsf{Tm}\,\mathsf{Nat}}}{\mathsf{ite}\,\mathsf{true}\,(\mathsf{num}\, 1)\,\mathsf{false} : \mathsf{Tm}\,\mathsf{Nat}}$$

Exercise 27. *Show that induction is equivalent to initiality.*

2.3.1 Type inference

The AST-level definition of Razor is the following.

Definition 28 (Untyped Razor). *A model comprises the following components:*

$$\begin{aligned}
\mathsf{Tm} &: \mathsf{Set}\\
\mathsf{true} &: \mathsf{Tm}\\
\mathsf{false} &: \mathsf{Tm}\\
\mathsf{ite} &: \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm}\\
\mathsf{num} &: \mathbb{N} \to \mathsf{Tm}\\
_ + _ &: \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm}\\
\mathsf{isZero} &: \mathsf{Tm} \to \mathsf{Tm}
\end{aligned}$$

We define the following model of untyped Razor referring to the syntax of (typed) Razor. (Actually any model with decidability of equality for Ty suffices.)

Definition 29 (Type inference model).

$$\begin{aligned}
\mathsf{Tm} &:= \mathsf{Maybe}\,((A : \mathsf{Ty}) \times \mathsf{Tm}\,A)\\
\mathsf{true} &:= \mathsf{just}\,(\mathsf{Bool}, \mathsf{true})\\
\mathsf{false} &:= \mathsf{just}\,(\mathsf{Bool}, \mathsf{false})\\
\mathsf{ite}\,t\,u\,v &:= \mathsf{match}\,(t,u,v)\,\{(\mathsf{just}\,(\mathsf{Bool}, t'), \mathsf{just}\,(\mathsf{Bool}, u'), \mathsf{just}(\mathsf{Bool}, v')) \mapsto \mathsf{just}\,(\mathsf{Bool}, \mathsf{ite}\,t'\,u'\,v');\\
&\qquad (\mathsf{just}\,(\mathsf{Bool}, t'), \mathsf{just}\,(\mathsf{Nat}, u'), \mathsf{just}(\mathsf{Nat}, v')) \mapsto \mathsf{just}\,(\mathsf{Nat}, \mathsf{ite}\,t'\,u'\,v');\\
&\qquad _ \mapsto \mathsf{nothing}\}\\
\mathsf{num}\,n &:= \mathsf{just}\,(\mathsf{Nat}, \mathsf{num}\,n)\\
u + v &:= \mathsf{match}\,(u,v)\,\{(\mathsf{just}\,(\mathsf{Nat}, u'), \mathsf{just}\,(\mathsf{Nat}, v')) \mapsto \mathsf{just}\,(\mathsf{Nat}, u' + v'); _ \mapsto \mathsf{nothing}\}\\
\mathsf{isZero}\,t &:= \mathsf{match}\,t\,\{\mathsf{just}\,(\mathsf{Nat}, t') \mapsto \mathsf{just}\,(\mathsf{Bool}, \mathsf{isZero}\,t'); _ \mapsto \mathsf{nothing}\}
\end{aligned}$$

Interpretation into this model is type inference. We apologise for the extreme overloading. It is clear from the absence of indices in the input of the function that we refer to the syntax of untyped Razor:

$$\mathsf{infer} : \mathsf{Tm}_{\mathsf{I}} \to \mathsf{Maybe}\,((A : \mathsf{Ty}) \times \mathsf{Tm}\,A)$$

For example, we compute

$$\begin{aligned}
&\mathsf{infer}\,(\mathsf{ite}\,(\mathsf{num}\,1)\,\mathsf{true}\,\mathsf{false}) =\\
&\mathsf{match}\,(\mathsf{infer}\,(\mathsf{num}\,1), \mathsf{infer}\,\mathsf{true}, \mathsf{infer}\,\mathsf{false})\,\{(\mathsf{just}\,(\mathsf{Bool}, t'), \mathsf{just}\,(\mathsf{Bool}, u'), \mathsf{just}(\mathsf{Bool}, v')) \mapsto \mathsf{just}\,(\mathsf{Bool}, \mathsf{ite}\,t'\,u'\,v');\\
&\qquad (\mathsf{just}\,(\mathsf{Bool}, t'), \mathsf{just}\,(\mathsf{Nat}, u'), \mathsf{just}(\mathsf{Nat}, v')) \mapsto \mathsf{just}\,(\mathsf{Nat}, \mathsf{ite}\,t'\,u'\,v');\\
&\qquad _ \mapsto \mathsf{nothing}\} =\\
&\mathsf{match}\,(\mathsf{just}\,(\mathsf{Nat}, \mathsf{num}\,1), \mathsf{just}\,(\mathsf{Bool}, \mathsf{true}), \mathsf{just}\,(\mathsf{Bool}, \mathsf{false}))\,\{\\
&\quad (\mathsf{just}\,(\mathsf{Bool}, t'), \mathsf{just}\,(\mathsf{Bool}, u'), \mathsf{just}(\mathsf{Bool}, v')) \mapsto \mathsf{just}\,(\mathsf{Bool}, \mathsf{ite}\,t'\,u'\,v');\\
&\quad (\mathsf{just}\,(\mathsf{Bool}, t'), \mathsf{just}\,(\mathsf{Nat}, u'), \mathsf{just}(\mathsf{Nat}, v')) \mapsto \mathsf{just}\,(\mathsf{Nat}, \mathsf{ite}\,t'\,u'\,v');\\
&\quad _ \mapsto \mathsf{nothing}\} =\\
&\mathsf{nothing}.
\end{aligned}$$

Exercise 30. *Show that* $\mathsf{infer}\,(\mathsf{num}\,1 + \mathsf{num}\,2) = \mathsf{just}\,(\mathsf{Nat}, (\mathsf{num}\,1 + \mathsf{num}\,2))$. *Show that there is an* A *and* $t : \mathsf{Tm}\,A$ *such that* $\mathsf{infer}\,(\mathsf{num}\,0 + (\mathsf{ite}\,(\mathsf{isZero}\,(\mathsf{num}\,1))\,(\mathsf{num}\,1)\,(\mathsf{num}\,2))) = \mathsf{just}\,(A, t)$.

infer (num 1 + num 2) = match (infer (num 1), infer (num 2)) {(just (Nat, u'), just (Nat, v')) ↦ just (Nat, u' + v); _ ↦ nothing} = match (just (Nat,num 1), just (Nat,num 2)) {(just (Nat, u'), just (Nat, v')) ↦ just (Nat, u' + v); _ ↦ nothing} = just (Nat, (num 1 + num 2))

A = Nat lesz, t pedig t = num 0 + (ite (isZero (num 1)) (num 1) (num 2)) = num 0 + num 2 = num 2.

Exercise 31. *Define type erasure, a map from well-typed terms to untyped terms* $\mathsf{erase} : \mathsf{Tm}\,A \to \mathsf{Tm}$. *Give examples of* $t : \mathsf{Tm}$ *such that there is no* A *and* $t' : \mathsf{Tm}\,A$ *with* $\mathsf{erase}t' = t$. *Show soundness of inference, that is,* $\mathsf{infer}\,t = \mathsf{just}\,(A, t')$ *implies* $\mathsf{erase}\,t' = t$. *Show completeness of inference, that is, for any* $t : \mathsf{Tm}\,A$, *we have* $\mathsf{infer}\,(\mathsf{erase}\,t) = \mathsf{just}\,(A, t)$. *Using soundness and/or completeness, give generic ways to prove statements such as "for a given* t, *show that (doesn't) exist* A *and* $t' : \mathsf{Tm}\,A$ *such that* $\mathsf{erase}\,t' = t$*".*

1. A típustörlés (erase**) definiálása**

A típustörlés egy jól típusozott termet (Tm A) alakít át típusozatlan absztrakt szintaxisfává (Tm) úgy, hogy minden típusinformációt eltávolít belőle. A szabályok a következők:

- erase true = true
- erase false = false
- erase (num n) = num n
- erase (ite b u v) = ite (erase b) (erase u) (erase v)
- erase (isZero t) = isZero (erase t)
- erase (u + v) = erase u + erase v.

2. Példák nem típusozható termekre

Vannak olyan típusozatlan termek, amelyekhez nem létezik olyan A típus és t' típusozott term, amelyből típustörléssel megkaphatóak. Ilyen például:

- isZero true: Az isZero művelet számot vár, de logikai értéket kapott.
- true + num 1: Az összeadás két számot vár, de az egyik paramétere logikai.
- ite (num 1) true false: A feltételnek logikainak kellene lennie, de itt egy szám szerepel.

3. Helyesség és teljesség

- **A típuskikövetkeztetés helyessége (Soundness):** Ha infer t = just (A, t'), az azt jelenti, hogy a kikövetkeztetett t' term típusa valóban A, és ha ebből töröljük a típusokat, visszakapjuk az eredeti t termet.
- **A típuskikövetkeztetés teljessége (Completeness):** Ha egy termnek létezik érvényes típusozása (vagyis létezik olyan t' : Tm A), akkor az infer algoritmus a típusozatlan változatból (erase t') kiindulva garantáltan megtalálja ezt az A típust és a t' termet.

4. Hogyan bizonyítható a típusozhatóság létezése vagy hiánya?

A fentiek alapján egy tetszőleges típusozatlan t termre:

- **Létezés bizonyítása:** Futtassuk le az infer t algoritmust. Ha az eredmény just (A, t'), akkor a helyesség tétele miatt megtaláltuk az érvényes típusozást.
- **Hiány bizonyítása:** Futtassuk le az infer t algoritmust. Ha az eredmény nothing, akkor a teljesség tétele miatt biztosak lehetünk benne, hogy nincs olyan A és t', amelyre erase t' = t teljesülne (hiszen, ha lenne, az algoritmus megtalálta volna).

🧸 Chapter 1: Levels of Abstraction

The big question: what IS a program?

Imagine you draw a **cat** 😺. Now imagine someone else draws the same cat, but slightly differently. Are they the same cat? Or different?

It depends on **what you care about**:

- If you care about exact pencil strokes → they're **different**
- If you care about the shape → maybe **the same**
- If you care about the meaning → they're **both just "a cat"**

Chapter 1 is about exactly this question, but for **programs**. A program can be looked at in **5 different ways**, from very messy and concrete, to very clean and abstract.

🪁 The toy language: Razor

The document uses a tiny pretend programming language called **Razor** to show the 5 levels. Razor has:

- **Numbers**: num 0, num 1, num 2, ...
- **Addition**: +
- **Zero check**: isZero — asks "is this number zero?"
- **If-then-else**: if ... then ... else ...

A Razor program looks like:

if isZero (num 0 + num 1) then false else isZero (num 0)

It means: *"Is 0+1 equal to zero? If yes → false. If no → is 0 zero?"*

The answer is true (because 0+1 = 1, which is not zero, so we go to the else branch, and isZero(0) = true).

The 5 levels 🪜

Think of these like **5 ways to look at the same thing**, each one cleaner than the last.

Level 1: 🔤 Just a string of characters

This is the most raw, messy level. The program is just letters and spaces typed on a keyboard:

"if isZero(num 0 + num 1)then false else isZero(num 0)"

Extra spaces, weird brackets — it's all just characters. The computer doesn't understand anything yet. It's like a baby looking at the word **CAT** but not knowing it means a furry animal.

Level 2: 🪙 A list of tokens

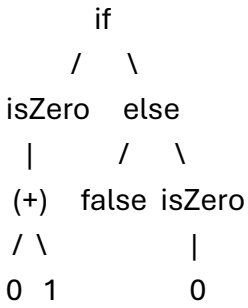
Now we split the string into **meaningful chunks** called tokens, and throw away the spaces:

[if, isZero, (, num, 0, +, num, 1,), then, false, else, isZero, (, num, 0,)]

We now know the individual words/symbols, but we still don't know **how they fit together**. It's like knowing the words in a sentence but not understanding the grammar yet.

Level 3: 🌳 A syntax tree

Now we arrange the tokens into a **tree** that shows how the pieces fit together. Like a family tree, but for your program!



Now we understand the **structure** — what belongs inside what. But there's still a problem: at this level, you could write if (num 0) then (num 1) else false — which mixes numbers and booleans in a nonsensical way. The tree allows it. It doesn't know any better.

Level 4: ✅ A typed syntax tree

Now we add **types**. Every piece of the program must have a type, and the types must **make sense together**.

Types in Razor are:

- Nat — a number (0, 1, 2, ...)
- Bool — true or false

Rules:

- num 0 has type Nat ✅
- isZero takes a Nat and gives a Bool ✅
- You can't do isZero false — false is a Bool, not a Nat ❌

So at level 4, **nonsense programs simply cannot exist**. You can only build programs where the types match up. It's like LEGO bricks where round holes only accept round pegs.

Level 5: 💎 Typed + quotiented (the cleanest level)

This is the most abstract level, and the one the **whole document uses**.

At level 4 we still distinguish between programs that **compute the same thing**. For example:

- num 0 + num 1 and num 1 are different trees at level 4
- But they **mean exactly the same thing** — they both equal 1

At level 5 we say: **if two programs always give the same answer, they ARE the same program.** We glue them together, or "quotient" them.

So at level 5:

- isZero (num 0) = true (they are literally the same thing)
- num 0 + num 0 = num 0

It's like saying 2+3 and 5 are not just *equal*, they are *the same number*. No difference at all.

🗺️ **Summary table**

Level	Name	What it is	Problem it removes
1	String	Raw characters	—
2	Token list	Meaningful words	Extra spaces
3	Syntax tree	Structure shown	Extra brackets
4	Typed tree	Types enforced	Nonsense programs
5	Typed + quotiented	Equal programs unified	Redundant computation

🔑 **The key message of Chapter 1**

The document says: **we will always work at Level 5.**

Why? Because at Level 5:

- Every term **automatically has a type** — you never need to check separately
- Evaluation is already **baked in** — isZero (num 0) and true are not just equal, they're *identical*
- There's nothing messy left to deal with — no accidental spaces, no silly brackets, no type mismatches, no redundant computations

This clean, maximally-abstract view is what makes the SOGAT framework so powerful — and it's the foundation for everything in Chapters 2, 3, 4, and 5.

3 Derivability in SOGATs

Second-order GATs (SOGATs) allow second-order operations. Second-order operations are also called binders. In this section, we define several languages with binders as SOGATs, and show how to define derivable operations and prove derivable equations in second-order models. The phrase "second-order" is a negative qualifier: second-order algebraic theories are not really algebraic (this is like people's democracy, which is not really democracy). For example, there is no good notion of morphism between second-order models. However, for derivability, second-order models are good enough. We will treat this problem in Section 4.

3.1 STLC (Simply Typed Lambda Calculus)

Definition 54 (Simply typed lambda calculus, STLC)**.** *A second-order model of STLC comprises the following components:*

$$\begin{aligned}
&\mathsf{Ty} && : \mathsf{Set}\\
&\mathsf{Tm} && : \mathsf{Ty} \to \mathsf{Set}\\
&\iota && : \mathsf{Ty}\\
&- \Rightarrow - && : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty}\\
&\mathsf{lam} && : (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \to \mathsf{Tm}\,(A \Rightarrow B)\\
&- \cdot - && : \mathsf{Tm}\,(A \Rightarrow B) \to \mathsf{Tm}\, A \to \mathsf{Tm}\, B\\
&\beta && : \mathsf{lam}\, b \cdot a = b\, a\\
&\eta && : f = \mathsf{lam}\, \lambda x.\, f \cdot x
\end{aligned}$$

The operators lam and $- \cdot -$ take *implicit arguments*, A and B. E.g. the explicit type of lam is $\{A : \mathsf{Ty}\}\{B : \mathsf{Ty}\} \to (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \to \mathsf{Tm}\,(A \Rightarrow B)$. All the arguments of the equations β and η are implicit. E.g. the explicit type of β is $\{A : \mathsf{Ty}\}\{B : \mathsf{Ty}\}\{b : \mathsf{Tm}\, A \to \mathsf{Tm}\, B\}\{a : \mathsf{Tm}\, A\} \to (- \cdot -)\,\{A\}\{B\}\,(\mathsf{lam}\,\{A\}\,\{B\}\, b)\, a = b\, a$. Note that in the equation η, f has to be in $\mathsf{Tm}\,(A \Rightarrow B)$ for some A and B for the equation to make sense.

The operation lam is second-order (also called a *binder*): its third input (after the two implicit inputs A, B) is a function.

Notation 55 (Isomorphism)**.** *The last four lines can be written more consisely:*

$$\mathsf{lam} : (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot -$$

We say that the sets $(\mathsf{Tm}\, A \to \mathsf{Tm}\, B)$ *and* $\mathsf{Tm}\,(A \Rightarrow B)$ *are isomorphic. We write the maps on the two sides of* $\cong$*, the notation in general is*

$$(f : X \cong Y : g) \;:=\; (f : X \to Y) \times (g : Y \to X) \times \big((x : X) \to g\,(f\, x) = x\big) \times \big((y : Y) \to f\,(g\, y) = y\big).$$

Derivability is programming: we can program in a SOGAT by postulating a second-order model in Agda or Coq and combining the components to build new programs. We can run the programs by proving (deriving) equalities between them.

Example 56. *In any second-order model of STLC, we define the identity function on* ι *by* $\mathsf{lam}\,\{\iota\}\{\iota\}\,(\lambda x.\, x)$*, which is in* $\mathsf{Tm}\,(\iota \Rightarrow \iota)$*. Note the difference between the object theory* lam *and meta* λ*. Actually, we can define the identity function on any type using meta quantification:*

$$\lambda A.\, \mathsf{lam}\,\{A\}\{A\}\,(\lambda x.\, x) : (A : \mathsf{Ty}) \to \mathsf{Tm}\,(A \Rightarrow A).$$

We compute (run the programs) in a second-order model deriving equalities. E.g. given $u : \mathsf{Tm}\,\iota$ *we argue*

$$(\mathsf{lam}\,(\lambda x.\, x)) \cdot u \overset{\beta}{=} (\lambda x.\, x)\, u \overset{\text{meta function application}}{=} u.$$

This seems like cheating: we avoided writing substitution by pushing it into the metatheory. But this is OK, we do not want to fully bootstrap STLC, we just want to define it. In the metatheory we of course assume (higher-order) functions, function application, its β *rule, and so on.*

Notation 57 (Derivation rules for SOGATs)**.** *Similarly to GATs, the derivation rule notation uses uncurried function space, named parameters and horizontal lines instead of the arrow* $\to$ *symbol. For arrows in second-order positions, we use the turnstile* $\vdash$*, and we use named function application for the* $\vdash$ *function space. Second-order models of STLC are described by the following derivation rules:*

$$\frac{}{\mathsf{Ty} : \mathsf{Set}} \qquad \frac{A : \mathsf{Ty}}{\mathsf{Tm}\, A : \mathsf{Set}} \qquad \frac{}{\iota : \mathsf{Ty}} \qquad \frac{A : \mathsf{Ty} \qquad B : \mathsf{Ty}}{A \Rightarrow B : \mathsf{Ty}} \qquad \frac{x : \mathsf{Tm}\, A \vdash t : \mathsf{Tm}\, B}{\mathsf{lam}\, x.t : \mathsf{Tm}\,(A \Rightarrow B)}$$

$$\frac{t : \mathsf{Tm}\,(A \Rightarrow B) \qquad a : \mathsf{Tm}\,A}{t \cdot a : \mathsf{Tm}\,B} \qquad \frac{}{(\mathsf{lam}\,x.b) \cdot a = b[x \mapsto a]}\,\beta \qquad \frac{}{f = \mathsf{lam}\,x.f \cdot x}\,\eta$$

Some arguments of the horizontal line function space can be implicit. Sometimes the name of the rule is written on the right hand side of the horizontal line.

We can omit the $: \mathsf{Ty}$ *and* Tm *parts and more concisely write the following without sacrificing precision (this is always the case when there are two sorts* $\mathsf{Ty} : \mathsf{Set}$, $\mathsf{Tm} : \mathsf{Ty} \to \mathsf{Set}$*).*

$$\bar{\iota} \qquad \frac{A \qquad B}{A \Rightarrow B} \qquad \frac{x : A \vdash t : B}{\mathsf{lam}\,x.t : A \Rightarrow B} \qquad \frac{t : A \Rightarrow B \qquad a : A}{t \cdot a : B} \qquad \frac{}{(\mathsf{lam}\,x.b) \cdot a = b[x \mapsto a]} \qquad \frac{}{f = \mathsf{lam}\,x.f \cdot x}$$

Example 58 (Second-order operations in mathematics)**.** *For the integral operation, the derivation rule notation is on the left and the second-order algebraic notation is on the right:*

$$\frac{a : \mathbb{R} \qquad b : \mathbb{R} \qquad x : \mathbb{R} \vdash t : \mathbb{R}}{\int_a^b t\,dx} \qquad\qquad \int : \mathbb{R} \to \mathbb{R} \to (\mathbb{R} \to \mathbb{R}) \to \mathbb{R},$$

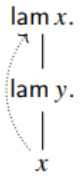
the expression $\int_0^1 \frac{1}{x^2}\,dx$ *corresponds to* $\int 0\,1\,(\lambda x.\,\frac{1}{x^2})$*.*

Notation 59 (Derivation trees for SOGATs)**.** *When we write derivation trees, the second-order arguments are collected on the left hand side of the* $\vdash$*. For example, given* $A : \mathsf{Ty}$ *and* $B : \mathsf{Ty}$*, we derive the constant function as follows: the verbose notation is on the left, the concise is on the right. In the concise notation we also omit the* λ*s.*

$$\frac{\dfrac{\overline{x : \mathsf{Tm}\,A, y : \mathsf{Tm}\,B \vdash x : \mathsf{Tm}\,A}}{x : \mathsf{Tm}\,A \vdash \mathsf{lam}\,\lambda y.x : \mathsf{Tm}\,(B \Rightarrow A)}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.x : \mathsf{Tm}\,(A \Rightarrow B \Rightarrow A)} \qquad\qquad \frac{\dfrac{\overline{x : A, y : B \vdash x : A}}{x : A \vdash \mathsf{lam}\,y.x : B \Rightarrow A}}{\mathsf{lam}\,x.\mathsf{lam}\,y.x : A \Rightarrow B \Rightarrow A}$$

Now the leaves of the tree can be assumptions from the left hand side of the turnstile in addition to derivation rules without arguments. The algebraic (or Coq/Agda) version of this is simply the conclusion $\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.x$*, or* $\mathsf{lam}\,(\lambda x.\mathsf{lam}\,(\lambda y.x))$ *with more brackets.*

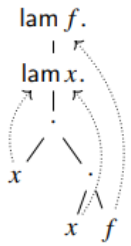
Notation 60 (Abstract binding trees (ABTs))**.** *Another notation for SOGAT-derivation trees is abstract* binding *trees where variables are pointers to the binders which are reachable through the path to the root. The constant function is drawn as follows.*



Abstract binding tree notation has the same issue as abstract syntax trees: the restriction that trees can only be put together in well-typed ways is not apparent.

Example 61. *We define the "double" function using short derivation rule and abstract binding tree notation.*

$$\dfrac{\dfrac{\overline{f : A \Rightarrow A, x : A \vdash f : A \Rightarrow A} \qquad \dfrac{\overline{f : A \Rightarrow A, x : A \vdash f : A \Rightarrow A} \qquad \overline{f : A \Rightarrow A, x : A \vdash x : A}}{f : A \Rightarrow A, x : A \vdash f \cdot x : A}}{f : A \Rightarrow A, x : A \vdash f \cdot (f \cdot x) : A}}{\dfrac{f : A \Rightarrow A \vdash \mathsf{lam}\,x.f \cdot (f \cdot x) : A \Rightarrow A}{\mathsf{lam}\,f.\mathsf{lam}\,x.f \cdot (f \cdot x) : (A \Rightarrow A) \Rightarrow A \Rightarrow A}}$$



Example 62. *Function composition:*

$$\dfrac{\dfrac{\overline{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash f : B \Rightarrow A} \qquad \dfrac{\overline{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g : C \Rightarrow B} \qquad \overline{\ldots, x : C \vdash x : C}}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash g \cdot x : B}}{f : B \Rightarrow A, g : C \Rightarrow B, x : C \vdash f \cdot (g \cdot x) : A}}{\dfrac{\dfrac{f : B \Rightarrow A, g : C \Rightarrow B \vdash \mathsf{lam}\,x.f \cdot (g \cdot x) : C \Rightarrow A}{f : B \Rightarrow A \vdash \mathsf{lam}\,g.\mathsf{lam}\,x.f \cdot (g \cdot x) : (C \Rightarrow B) \Rightarrow C \Rightarrow A}}{\mathsf{lam}\,f.\mathsf{lam}\,g.\mathsf{lam}\,x.f \cdot (g \cdot x) : (B \Rightarrow A) \Rightarrow (C \Rightarrow B) \Rightarrow C \Rightarrow A}}$$

Example 63. *We define a model of STCC (Definition 32) assuming a second-order model of STLC.*

$$\begin{aligned}
\mathsf{Ty} &:= \mathsf{Ty}\\
\mathsf{Tm}\,A &:= \mathsf{Tm}\,A\\
\iota &:= \iota\\
A \Rightarrow B &:= A \Rightarrow B\\
\mathsf{K} &:= \mathsf{lam}\,(\lambda u.\,\mathsf{lam}\,(\lambda f.\,u))\\
\mathsf{S} &:= \mathsf{lam}\left(\lambda f.\,\mathsf{lam}\,(\lambda g.\,\mathsf{lam}\,(\lambda u.\,f\cdot u\cdot(g\cdot u)))\right)
\end{aligned}$$

The equations hold, e.g.:

$$\begin{aligned}
\mathsf{K}\beta : \mathsf{K}\cdot u\cdot f &\quad =(\textit{definition of } \mathsf{K})\\
\left(\left(\mathsf{lam}\,(\lambda u.\,\mathsf{lam}\,(\lambda f.\,u))\right)\cdot u\right)\cdot f &\quad =(\beta)\\
\left((\lambda u.\,\mathsf{lam}\,(\lambda f.\,u))\,u\right)\cdot f &\quad =(\textit{meta function application})\\
\left(\mathsf{lam}\,(\lambda f.\,u)\right)\cdot f &\quad =(\beta)\\
(\lambda f.\,u)\,f &\quad =(\textit{meta function application})\\
u &
\end{aligned}$$

Exercise 64. *Prove that* $\mathsf{S}\beta$ *holds.*

Sβ : S · f · g · u = (definition of S)

(lam (λf. lam (λg. lam (λu. f · u · (g · u))))) · f · g · u = (β)

(lam (λg. lam (λu. f · u · (g · u)))) · g · u = (β)

(lam (λu. f · u · (g · u))) · u = (β)

f · u · (g · u)

Food for thought 65. *Can we derive a second-order model of STLC from a model of STCC? What if we start with a second-order model having some extra equations [AKSV23]?*

Example 66. *We can use derivation tree building to do informal type inference. For example, we don't know whether the expression*

$$\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z$$

has a type (makes sense in a second-order model of STLC), so we try to build its derivation tree from the bottom. Let's say it has an arbitrary type A.

$$\frac{?}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : A}$$

As the expression starts with lam*, we use its derivation rule and we learn that* $A = B \Rightarrow C$ *for some* B *and* C*:*

$$\frac{\dfrac{?}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : C}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow C}$$

We use lam *again and we learn* $C = D \Rightarrow E$*, so we replace all occurrences of* C *with* $D \Rightarrow E$*:*

$$\frac{\dfrac{\dfrac{?}{x : B, y : D \vdash \mathsf{lam}\,\lambda z.y\cdot x\cdot z : E}}{x : B \vdash \mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : D \Rightarrow E}}{\mathsf{lam}\,\lambda x.\mathsf{lam}\,\lambda y.\mathsf{lam}\,\lambda z.y\cdot x\cdot z : B \Rightarrow (D \Rightarrow E)}$$

We use lam *again and we learn* $E = F \Rightarrow G$*:*

$$\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

Now we can only use the derivation rule of application $(\cdot)$*, and we introduce a new metavariable* H *because we cannot read off the domain of the function from its conclusion. Recall that* $y \cdot x \cdot z = (y \cdot x) \cdot z$*.*

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y \cdot x : H \Rightarrow G} \qquad \dfrac{?}{x : B, y : D, z : F \vdash z : H}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

We first reach a leaf on the right hand side: z *is to the left of the turnstile, so we learn* $H = F$*:*

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G} \qquad \dfrac{}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

On the left hand side, there is an application $\cdot$*, we introduce a new metavariable* I*:*

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y : I \Rightarrow F \Rightarrow G} \qquad \dfrac{?}{x : B, y : D, z : F \vdash x : I}}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G} \qquad \dfrac{}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

The solution to the right hand side $?$ *forces* $I = B$*, because this is the type of* x*:*

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{?}{x : B, y : D, z : F \vdash y : B \Rightarrow F \Rightarrow G} \qquad \dfrac{}{x : B, y : D, z : F \vdash x : B}}{x : B, y : D, z : F \vdash y \cdot x : F \Rightarrow G} \qquad \dfrac{}{x : B, y : D, z : F \vdash z : F}}{x : B, y : D, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : D \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : D \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow (D \Rightarrow (F \Rightarrow G))}$$

On the left hand side we learn $D = B \Rightarrow F \Rightarrow G$*:*

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y : B \Rightarrow F \Rightarrow G} \qquad \dfrac{}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash x : B}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x : F \Rightarrow G} \qquad \dfrac{}{\ldots, z : F \vdash z : F}}{x : B, y : B \Rightarrow F \Rightarrow G, z : F \vdash y \cdot x \cdot z : G}}{x : B, y : B \Rightarrow F \Rightarrow G \vdash \mathsf{lam}\, \lambda z.y \cdot x \cdot z : F \Rightarrow G}}{x : B \vdash \mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : (B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G)}}{\mathsf{lam}\, \lambda x.\mathsf{lam}\, \lambda y.\mathsf{lam}\, \lambda z.y \cdot x \cdot z : B \Rightarrow ((B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G))}$$

So the most general type of the expression is $B \Rightarrow ((B \Rightarrow F \Rightarrow G) \Rightarrow (F \Rightarrow G))$ *for some types* B*,* F*,* G*.*

Exercise 67. *Using the above method, try to infer the type of the following epxressions (we assume Razor extended with functions merging Definitions 14 and 54):*

$$\mathsf{lam}\,\lambda x.\, x \cdot x$$
$$\mathsf{lam}\,\lambda f.\, f \cdot (\mathsf{num}\,1 + f \cdot \mathsf{true})$$
$$\mathsf{lam}\,\lambda f.\, f \cdot (\mathsf{num}\,1 + f \cdot \mathsf{num}\,2)$$

A lényeg: az első nem típusozható, mert x önmagára van alkalmazva. A második sem, mert az f · true miatt Bool a bemenet, viszont a kimenetnek Nat-nak kéne lennie, azt adnánk hozzá a num 1-hez. A harmadik viszont már típusozható és ((Nat ⇒ Nat) ⇒ Nat) lesz, mert ott f egy Natot váró és adó függvény, ami pont, hogy Natokra (num 2, valamint num 1 + f · num 2 eredménye) van alkalmazva :D

$$\frac{?}{\mathsf{lam}\ \lambda x.\ x \cdot x : A} \qquad \frac{\dfrac{?}{x : B \vdash x \cdot x : C}}{\mathsf{lam}\ \lambda x.\ x \cdot x : B \Rightarrow C} \qquad \frac{\dfrac{\dfrac{?}{x : B \vdash x : D \Rightarrow C} \quad \dfrac{?}{x : B \vdash x : D \Rightarrow C}}{x : B \vdash x \cdot x : C}}{\mathsf{lam}\ \lambda x.\ x \cdot x : B \Rightarrow C}$$

Exercise 68. *Show that function composition (Example 62) is associative. That is, abbreviating* $\mathsf{comp} := \mathsf{lam}\, f.\mathsf{lam}\, g.\mathsf{lam}\, x.f \cdot (g \cdot x)$*, for every* u*,* v*,* w *we have* $\mathsf{comp} \cdot (\mathsf{comp} \cdot u \cdot v) \cdot w = \mathsf{comp} \cdot u \cdot (\mathsf{comp} \cdot v \cdot w)$.

Azt kell belátni, hogy három függvény kompozíciójánál mindegy, hogy melyik kettőt komponáljuk először és melyiket harmadjára.

Baloldal:

comp · (comp · u · v) · w = (definíció)
comp · (((lam f. lam g. lam x. f · (g · x)) · u) · v) · w = (β)
comp · ((lam g. lam x. u · (g · x)) · v) · w = (β)
comp · (lam x. u · (v · x)) · w = (definíció)
(lam f. lam g. lam y. f · (g · y)) · (lam x. u · (v · x)) · w = (β)
(lam g. lam y. (lam x. u · (v · x)) · (g · y)) · w = (β)
lam y. (lam x. u · (v · x)) · (w · y) = (β)
<u>lam y. u · (v · (w · y))</u>

Jobboldal:

comp · u · (comp · v · w) = (definíció)
comp · u · ((lam f. lam g. lam x. f · (g · x)) · v · w) = (β)
comp · u · ((lam g. lam x. v · (g · x)) · w) = (β)
comp · u · (lam x. v · (w · x)) = (definíció)
(lam f. lam g. lam y. f · (g · y)) · u · (lam x. v · (w · x)) = (β)
(lam f. lam g. lam y. f · (g · y)) · u · (lam x. v · (w · x)) = (β)
(lam g. lam y. u · (g · y)) · (lam x. v · (w · x)) = (β)
lam y. u · ((lam x. v · (w · x)) · y) = (β)
<u>lam y. u · (v · (w · y))</u>

Mindkét oldalon ugyanaz jött ki, tehát teljesül.

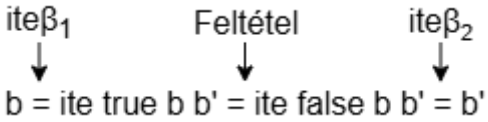
Exercise 69. *Show that for any* $A : \mathsf{Ty}$, $\mathsf{Tm}\,(A \Rightarrow A)$ *is a monoid with function composition and identity.*

Exercise 70. *Show that we can define a category where objects are* Ty, *a morphism from* A *to* B *is* $\mathsf{Tm}\,(A \Rightarrow B)$, *composition is function composition (a refined version of the previous exercise).*

We cannot prove logical consistency just assuming a second-order model, because it might be the trivial model (where all sorts are $\mathbb{1}$). We also cannot prove equational consistency saying that there are terms a, a' such that $a = a' \to \mathbb{0}$, but we can get something similar: instead of obtaining a meta $\mathbb{0}$, we obtain that the second-order model is trivial:

Exercise 71. *Prove that for any* $B : \mathsf{Ty}$ *we have a* $A : \mathsf{Ty}$ *and* $a, a' : \mathsf{Tm}\,A$ *where* $a = a'$ *implies that any* $b, b' : \mathsf{Tm}\,B$, $b = b'$.

Legyen A a Bool típus, a := true, a' := false. A feltevés szerint true = a = a' = false. A B típus lehet ugyanúgy Bool, vagy akár Nat is, a levezetés hasonló lesz egy már korábbihoz:



Exercise 72. *There is a second-order model of STLC where there is an* $A : \mathsf{Ty}$ *and* $a, a' : \mathsf{Tm}\,A$ *such that* $a \neq a'$.

A standard modell megfelel ennek. Ha A = Bool, a = true / tt és a' = false / ff. Értelemszerűen itt nem lesz egyenlő kettő, tehát elvileg teljesül, amit a feladat kér.

🌟 What's new in Chapter 3?

Remember Chapter 2? All the languages there (Razor, Monoid, STCC) had **no variables** — no way to write "let x be something and use it later."

Chapter 3 is where things get exciting: we now study languages **with binders** — operations that introduce variables. The simplest and most important example is the **lambda calculus**, which is the mathematical foundation of every functional programming language (Haskell, OCaml, etc.).

🔑 Definition 54: What is STLC?

A second-order model of STLC has these pieces:

The types

Ty : Set -- a collection of types

ι : Ty -- one base type (like "int")

A ⇒ B : Ty -- function types ("A arrow B")

Think of types like **shapes of boxes**:

- ι is a plain box 📦
- A ⇒ B is a machine 🏭 that takes an A-box in and gives a B-box out

The terms

Tm : Ty → Set -- terms of each type

lam : (Tm A → Tm B) → Tm (A ⇒ B) -- make a function

_·_ : Tm (A ⇒ B) → Tm A → Tm B -- apply a function

- lam **creates** a function. You give it a rule "if I have an x of type A, here's how to make a B" — and it wraps that into a term.
- _·_ **uses** a function. Give a function and an input, get an output.

The rules

β : lam b · a = b a -- run the function

η : f = lam λx. f · x -- every function IS a lam

- **β (beta)**: applying a lam to an argument just plugs the argument in. (lam λx. x+1) · 3 = 3+1. This is **how functions run**.
- **η (eta)**: every function of type A ⇒ B can be written as a lam. There's no "hidden" function that isn't a lambda. This is **uniqueness**.

📝 **Notation 55: The Isomorphism**

The document writes the last four lines more concisely as:

lam : (Tm A → Tm B) ≅ Tm (A ⇒ B) : _·_

The set of meta-functions from Tm A to Tm B is isomorphic to the set of terms of type A ⇒ B.

🤝 **What is an isomorphism?**

An isomorphism between two sets X and Y means:

- There's a function f : X → Y (going forward)
- There's a function g : Y → X (going backward)
- g(f(x)) = x for all x (going forward then back = doing nothing)
- f(g(y)) = y for all y (going back then forward = doing nothing)

It means the two sets are **perfectly equivalent** — same size and structure, just with different labels.

So here:

- lam goes **forward**: takes a meta-function, gives a term
- _·_ goes **backward**: takes a term, gives a meta-function (by fixing the left argument)
- The β rule says: (lam b) · a = b a → go forward then backward = identity ✅
- The η rule says: f = lam (λx. f · x) → go backward then forward = identity ✅

In short: **a function in the language IS exactly a meta-function**, and vice versa. There's no difference between the two.

🔑 What makes lam "second-order"?

In Razor, all operations took **terms** as input:

isZero : Tm Nat → Tm Bool

But lam takes a **function** (from terms to terms) as input:

lam : (Tm A → Tm B) → Tm (A ⇒ B)

A function from terms to terms is a **second-order** input — it operates *on* terms, not just *is* a term. This is what makes STLC a **SOGAT** rather than a GAT.

Think of it this way:

- First-order: "give me a number" → you hand over 5
- Second-order: "give me a rule for turning numbers into numbers" → you hand over λx. x+1

lam accepts that **rule** and packages it into a term. That rule is the **binder** — it binds the variable x inside the body.

🎮 Derivation rules notation (Notation 57)

The document also writes STLC using the familiar **derivation rule** notation:

$$\frac{x : A \vdash t : B}{\text{lam } x.t : A \Rightarrow B}$$

The ⊢ (turnstile) is the new thing for SOGATs. It means: **"assuming x has type A, t has type B."** The left side of ⊢ is a list of **variable assumptions** — this is where binders live.

Compare:

- In GATs: A → B (first-order, no variables)
- In SOGATs: x : A ⊢ t : B (second-order, with a bound variable)

📚 Key examples from the document

The identity function (Example 56)

lam {A}{A} (λx. x) : Tm (A ⇒ A)

This is the function "take any x, give it back." And it runs:

(lam λx. x) · u =β= (λx. x) u = u ✅

The constant function (Notation 59)

lam x. lam y. x : A ⇒ B ⇒ A

"Take x, then take y, return x (ignoring y)." This is K from STCC!

The double function (Example 61)

lam f. lam x. f · (f · x) ： (A ⇒ A) ⇒ A ⇒ A

"Take a function f and apply it twice to x."

Function composition (Example 62)

lam f. lam g. lam x. f · (g · x) ： (B ⇒ A) ⇒ (C ⇒ B) ⇒ C ⇒ A

🔗 **STLC generalises STCC (Example 63)**

The document shows that **STCC is derivable from STLC**! Given any STLC model, you can build a STCC model by defining:

K := lam λu. lam (λf. u)

S := lam λf. lam λg. lam (λu. f · u · (g · u))

And the K and S equations hold by β. So STLC is **strictly more powerful** than STCC — it can express everything STCC can and more (like arbitrary function definitions with variables).

🗺️ **Summary**

Concept	Meaning
ι	A base type (like int)
A ⇒ B	Function type from A to B
lam	Create a function by binding a variable
_·_	Apply a function to an argument
β rule	Running a function: plug in the argument
η rule	Every term of function type is a lam
Isomorphism	lam and _·_ are inverse to each other
Second-order	lam takes a *function* as input (not just a term)

3.2 System T

The following language has a proper type of natural numbers, it does not simply inherit $\mathbb{N}$ from the metatheory, like Razor did.

Definition 76 (System T)**.** *A second-order model of System T comprises the following components:*

$$
\begin{array}{ll}
\mathsf{Ty} & : \mathsf{Set} \\
\mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\
- \Rightarrow - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{lam} & : (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \cong \mathsf{Tm}\, (A \Rightarrow B) : - \cdot - \\
\mathsf{Nat} & : \mathsf{Ty} \\
\mathsf{zero} & : \mathsf{Tm}\, \mathsf{Nat} \\
\mathsf{suc} & : \mathsf{Tm}\, \mathsf{Nat} \to \mathsf{Tm}\, \mathsf{Nat} \\
\mathsf{ite} & : \mathsf{Tm}\, A \to (\mathsf{Tm}\, A \to \mathsf{Tm}\, A) \to \mathsf{Tm}\, \mathsf{Nat} \to \mathsf{Tm}\, A \\
\mathsf{Nat}\beta_1 & : \mathsf{ite}\, z\, s\, \mathsf{zero} = z \\
\mathsf{Nat}\beta_2 & : \mathsf{ite}\, z\, s\, (\mathsf{suc}\, t) = s\, (\mathsf{ite}\, z\, s\, t)
\end{array}
$$

Natural numbers are now given by zero and successor suc, for example, three is suc (suc (suc zero)). Functions out of natural numbers can be defined via ite (iterator, also called recursor, fold).

Definition 77 (Constructors, destructors, computation and uniqueness rules)**.** *Operators for each type can be grouped into constructors (introduction rules) and destructors (elimination rules, eliminators). Equations can be grouped into β (computation) and η (uniqueness) rules. β rules explain how to compute if a destructor is applied to a constructor, η rules express what happens if a constructor is applied to a destructor.*

We group our rules for System T and Razor for comparison.

language	type	constructor	destructor	computation	uniqueness
System T (Def. 76)	$- \Rightarrow -$	lam	$- \cdot -$	$\Rightarrow\beta$	$\Rightarrow\eta$
	Nat	zero, suc	ite	$\mathsf{Nat}\beta_1, \mathsf{Nat}\beta_2$	
Razor (Def. 14)	Bool	true, false	ite	$\mathsf{ite}\beta_1, \mathsf{ite}\beta_2$	
	Nat	num	$- + -$, isZero	$+\beta, \mathsf{isZero}\beta_1, \mathsf{isZero}\beta_2$	

Nat does not come with an η rule, but this will be fixed in Section 3.5.

Example 78. *The double function:*

$$
\begin{aligned}
&\mathsf{double} : \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat}) \\
&\mathsf{double} := \mathsf{lam}\, \lambda x.\mathsf{ite}\, \mathsf{zero}\, (\lambda y.\mathsf{suc}\, (\mathsf{suc}\, y))\, x
\end{aligned}
$$

Computing the double of 2:

$$
\begin{array}{ll}
\mathsf{double} \cdot \mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}) & =(\textit{abbreviation}) \\
(\mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x) \cdot \mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}) & =(\beta) \\
(\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x)\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})) & =(\textit{meta application}) \\
\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})) & =(\mathsf{Nat}\beta_2) \\
(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{suc}\,\mathsf{zero})) & =(\textit{meta aplication}) \\
\mathsf{suc}\,\Big(\mathsf{suc}\,\big(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{suc}\,\mathsf{zero})\big)\Big) & =(\mathsf{Nat}\beta_2) \\
\mathsf{suc}\,\Big(\mathsf{suc}\,\big((\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,\mathsf{zero})\big)\Big) & =(\textit{meta application}) \\
\mathsf{suc}\,\Big(\mathsf{suc}\,\big(\mathsf{suc}\,(\mathsf{suc}\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,\mathsf{zero}))\big)\Big) & =(\mathsf{Nat}\beta_1) \\
\mathsf{suc}\,\Big(\mathsf{suc}\,\big(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero})\big)\Big) &
\end{array}
$$

Definition 79 (Embedding $\mathbb{N}$ into $\mathsf{Tm}\,\mathsf{Nat}$)**.** *The meta function* $\ulcorner - \urcorner : \mathbb{N} \to \mathsf{Tm}\,\mathsf{Nat}$ *is defined as* $\ulcorner n \urcorner := \mathsf{suc}^n\,\mathsf{zero}$*, e.g.* $\ulcorner 3 \urcorner = \mathsf{suc}\,(\mathsf{suc}\,(\mathsf{suc}\,\mathsf{zero}))$.

Definition 80 (Definability)**.** *An* $f : \mathbb{N} \to \mathbb{N}$ *is definable in a second-order model of System T if there is a* $t : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat})$ *such that for all* n, $\ulcorner f\,n \urcorner = t \cdot \ulcorner n \urcorner$*. An* $f : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ *is definable if there is a* $t : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat} \Rightarrow \mathsf{Nat})$ *such that for all* m, n, $\ulcorner f\,m\,n \urcorner = t \cdot \ulcorner m \urcorner \cdot \ulcorner n \urcorner$.

Exercise 81. *Addition, multiplication, exponentiation, the Ackermann function are definable in any second-order model of System T.*

add := lam λm. lam λn. ite m suc n

multiply := lam λm. lam λn. ite zero (lam λr. add · m · r) n

exponent := lam λm. lam λn. ite (suc zero) (lam λr. multiply · m · r) n

Ackermann: minden m-re definiálni kell egy függvényt, aztán ezekből rakható össze...

Exercise 82. *Which of the following terms define identity?*

$$
\begin{array}{l}
\mathsf{lam}\,\lambda x.x \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,\mathsf{suc}\,x \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,x\,(\lambda y.y)\,x \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,\mathsf{zero}\,(\lambda y.y)\,x \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,\mathsf{zero})\,\mathsf{suc}\,x \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,\mathsf{zero})\,(\lambda y.y)\,x
\end{array}
$$

1-es, 2-es, és 3-as definiálnak identitást.

Exercise 83. *Which of the following terms define the function* $x \mapsto 2 * x + 1$*?*

$$
\begin{array}{l}
\mathsf{lam}\,\lambda x.\mathsf{suc}\,x \\
\mathsf{lam}\,\lambda x.\mathsf{suc}\,(\mathsf{ite}\,x\,\mathsf{suc}\,x) \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,x)\,\mathsf{suc}\,x \\
\mathsf{lam}\,\lambda x.\mathsf{ite}\,(\mathsf{suc}\,\mathsf{zero})\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x \\
\mathsf{lam}\,\lambda x.\mathsf{suc}\,(\mathsf{ite}\,\mathsf{zero}\,(\lambda y.\mathsf{suc}\,(\mathsf{suc}\,y))\,x)
\end{array}
$$

A 2-es, 3-as, 4-es és az 5-ös definálják a megadott függvényt, tehát csak az 1-es nem.

Exercise 84. *Show that if* $\mathsf{zero} = \mathsf{suc}\, t$*, then for any* $u, v : \mathsf{Tm}\, A$*, we have* $u = v$*.*

Exercise 85. *Show that the predecessor function is definable (difficult, but it will become easier once we have product types).*

Legyen f := ite u (λx. v) : Tm Nat → Tm A. Ekkor Natβ1 miatt f zero = ite u (λx. v) zero = u lesz, Natβ2 miatt pedig f (suc t) = ite u (λx. v) (suc t) = (λx. v)(ite u (λx. v) t) = v lesz. Tehát… u = f zero = f (suc t) = v

A pred elvileg nem definiálható csak az ite-t használva, akövetkező oldalon van is erről szó. Egy lehetséges megoldás párokat használni, ahol a második elem lesz, ami kell. Vagy marad a rec…

It is not possible to define a term $\mathsf{pred} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat})$ such that $\mathsf{pred} \cdot \mathsf{suc}\, t = t$ for any $t : \mathsf{Tm}\,\mathsf{Nat}$. [Par89]. For this, we would need the recursor for Nat which in its method for successor receives the natural number, not only the result of the recursive call:

$$\begin{aligned}
&\mathsf{rec} : \mathsf{Tm}\, A \to (\mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\, A\\
&\mathsf{rec}\, z\, s\, \mathsf{zero} = z\\
&\mathsf{rec}\, z\, s\, (\mathsf{suc}\, t) = s\, t\, (\mathsf{rec}\, z\, s\, t)
\end{aligned}$$

Now we can define $\mathsf{pred} := \mathsf{lam}\, \lambda x.\, \mathsf{rec}\, \mathsf{zero}\, (\lambda n\, _.\, n)\, x$ and it satisfies

$$\begin{aligned}
&\mathsf{pred} \cdot \mathsf{suc}\, t && =(\text{abbreviation})\\
&(\mathsf{lam}\, \lambda x.\mathsf{rec}\, \mathsf{zero}\, (\lambda y\, _.y)\, x) \cdot \mathsf{suc}\, t && =(\Rightarrow\beta)\\
&(\lambda x.\mathsf{rec}\, \mathsf{zero}\, (\lambda y\, _.y)\, x)\, (\mathsf{suc}\, t) && =(\text{meta application})\\
&\mathsf{rec}\, \mathsf{zero}\, (\lambda y\, _.y)\, (\mathsf{suc}\, t) && =(\mathsf{Nat}\beta_2)\\
&(\lambda y\, _.y)\, t\, (\mathsf{rec}\, \mathsf{zero}\, (\lambda y\, _.y)\, t) && =(\text{meta application})\\
&t.
\end{aligned}$$

Exercise 86. *Show that the iterator can be derived from the recursor.*

Itt elég figyelmen kívül hagyni az aktuális számot az iterálásban, amit a rekurzor biztosít. Tehát:

ite z s n := rec z (λx y. s y) n

Exercise 87. *Show that the factorial is definable in System T with recursor.*

factorial := lam λn. rec (suc zero) (lam λk. lam λr. multiply · suc(k) · r) n

Exercise 88. *Define a model of Razor (Definition 14) assuming a second-order model of System T with recursor.*

Ehhez meg kell adni mindent a Razorből a System T eszköztárát használva:

Razor komponens	**System T-s definíció**
Ty	Ty (ugyanaz)
Tm A	Tm A (ugyanaz)
Bool	Nat (a Bool értékei Nat értékkek kódolhatóak)
Nat	Nat (ugyanaz)
true	suc zero (= 1)
false	zero (= 0)
ite b u v	ite v (λx. u) b
num n	$\ulcorner n \urcorner$ = suc^n zero
u + v	ite u suc v
isZero u	ite (suc zero) (λx. zero) u

Meg lehet még nézni a β-szabályok teljesülését is még levezetéssel, hogy tényleg megfelelő-e így.

Theorem 89. *There is an* $\mathbb{N} \to \mathbb{N} \to \mathbb{N}$ *function, and if this function is definable in System T, then there is a* $v : \mathsf{Tm}\,\mathsf{Nat}$, *such that* $v = \mathsf{suc}\,v$.

Proof. First we fix a Gödel coding which takes an arbitrary term of type $\mathsf{Nat} \Rightarrow \mathsf{Nat}$ and turns it into a natural number: $\mathsf{code} : \mathsf{Tm}\,(\mathsf{Nat} \Rightarrow \mathsf{Nat}) \to \mathbb{N}$.

An evaluator for System T is an $\mathsf{eval} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ function with the following property: for every u and n, we have $\ulcorner \mathsf{eval}\,(\mathsf{code}\,u)\,n \urcorner = u \cdot \ulcorner n \urcorner$. Now eval being definable means that there is a t with $\ulcorner \mathsf{eval}\,m\,n \urcorner = t \cdot \ulcorner m \urcorner \cdot \ulcorner n \urcorner$ for every m and n. Now we define u as follows: $u := \mathsf{lam}\,\lambda x.\mathsf{suc}\,(t \cdot x \cdot x)$. And we reason:

$$\begin{aligned}
&t \cdot \ulcorner \mathsf{code}\,u \urcorner \cdot \ulcorner \mathsf{code}\,u \urcorner && =(\text{eval definable})\\
&\ulcorner \mathsf{eval}\,(\mathsf{code}\,u)\,(\mathsf{code}\,u) \urcorner && =(\text{property of eval})\\
&u \cdot \ulcorner \mathsf{code}\,u \urcorner && =(\text{definition of } u)\\
&(\mathsf{lam}\,\lambda x.\mathsf{suc}\,(t \cdot x \cdot x)) \cdot \ulcorner \mathsf{code}\,u \urcorner && =(\Rightarrow\beta)\\
&\mathsf{suc}\,(t \cdot \ulcorner \mathsf{code}\,u \urcorner \cdot \ulcorner \mathsf{code}\,u \urcorner)
\end{aligned}$$

So we choose $v := t \cdot \ulcorner \mathsf{code}\,u \urcorner \cdot \ulcorner \mathsf{code}\,u \urcorner$ and then $v = \mathsf{suc}\,v$. □

Remark 90. *Unlike* $\mathsf{zero} = \mathsf{suc}\,\mathsf{zero}$, *the existence of a* v *with* $v = \mathsf{suc}\,v$ *in a second-order model does not imply that any two terms are equal. However, the above result implies that an evaluator for System T cannot be defined in the syntax of System T. For the notion of syntax for second-order languages (like System T), see Section 4. If we have* $\mathsf{Nat}\eta$ *in a second-order model, then the existence of* v *with* $v = \mathsf{suc}\,v$ *implies that any two terms are equal. However, in the syntax of System T with* $\mathsf{Nat}\eta$, *equality of terms is undecidable, and thus a Gödel-coding function is not possible.*

Imagine you have a bag of building blocks. The type **Nat** is the *kind* "counting block". You can't hold a kind in your hands — you hold *actual blocks*.

zero is an empty hand — no blocks. **suc n** means "one more block than n". So three blocks is suc(suc(suc zero)).

The function type A ⇒ B is a *machine*: you feed it something of type A and it spits out something of type B. lam builds a machine, and · (application) feeds something into a machine.

ite is the counting machine. It takes three things:

1. A **starting value** z (the initial block)
2. A **step function** s (a rule: "given a result so far, what's the next result?")
3. A **number** n (how many steps to take)

It just **runs the step function n times**, starting from z: ite z s 3 = s(s(s(z))) — apply s three times to z.

The two computation rules are just saying this clearly:

- **Nat β1**: If you take 0 steps, you get back exactly z. Nothing happened.
- **Nat β2**: If you take suc n steps, it's the same as taking n steps and then doing one more s.

The big idea of System T is: **you can define any function on natural numbers that always terminates** by choosing the right z and s. Addition? Multiplication? Even the Ackermann function? All definable. But you can *not* define a function that might run forever.

The **recursor rec** is a stronger version of ite. The difference: ite's step s only sees the *result so far*, while rec's step also sees **what number we're currently at**. This extra information lets you define things like predecessor.

From **Definition 76**, a second-order model of System T has:

Component	Type	Meaning
Ty : Set	—	Types
Tm : Ty → Set	—	Terms of a type
– ⇒ –	Ty → Ty → Ty	Function type
lam / – · –	isomorphism	Lambda / Application
Nat	Ty	Natural number type
zero	Tm Nat	Zero
suc	Tm Nat → Tm Nat	Successor
ite	Tm A → (Tm A → Tm A) → Tm Nat → Tm A	Iterator

With rules:

- **Nat β1**: ite z s zero = z
- **Nat β2**: ite z s (suc t) = s (ite z s t)

The notation $\ulcorner n \urcorner := \mathrm{suc}^n$ zero (Definition 79) embeds meta-natural numbers into Tm Nat.

The **recursor** (introduced after Exercise 85):

rec : Tm A → (Tm Nat → Tm A → Tm A) → Tm Nat → Tm A

rec z s zero = z

rec z s (suc t) = s t (rec z s t)

It differs from ite in that the step s also receives the *current natural number* t.

3.3 Finite types

We extend STLC with nullary and binary sums and products.

Definition 91 (Language with finite types)**.** *A second-order model of Fin is STLC (Definition 54) extended*

with the following components:

$$
\begin{array}{ll}
\bot & : \mathsf{Ty} \\
\mathsf{exfalso} & : \mathsf{Tm}\,\bot \to \mathsf{Tm}\,A \\
\bot\eta & : (t : \mathsf{Tm}\,\bot \to \mathsf{Tm}\,A) \to t = \mathsf{exfalso} \\
- + - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{inl} & : \mathsf{Tm}\,A \to \mathsf{Tm}\,(A + B) \\
\mathsf{inr} & : \mathsf{Tm}\,B \to \mathsf{Tm}\,(A + B) \\
\mathsf{case} & : (\mathsf{Tm}\,A \to \mathsf{Tm}\,C) \to (\mathsf{Tm}\,B \to \mathsf{Tm}\,C) \to \mathsf{Tm}\,(A + B) \to \mathsf{Tm}\,C \\
+\beta_1 & : \mathsf{case}\,f\,g\,(\mathsf{inl}\,a) = f\,a \\
+\beta_2 & : \mathsf{case}\,f\,g\,(\mathsf{inr}\,b) = g\,b \\
+\eta & : (t : \mathsf{Tm}\,(A + B) \to \mathsf{Tm}\,C) \to t = \mathsf{case}\,(\lambda a.\,t\,(\mathsf{inl}\,a))\,(\lambda b.\,t\,(\mathsf{inr}\,b)) \\
\top & : \mathsf{Ty} \\
\mathsf{tt} & : \mathsf{Tm}\,\top \\
\top\eta & : t = tt \\
- \times - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\
-, - & : \mathsf{Tm}\,A \to \mathsf{Tm}\,B \to \mathsf{Tm}\,(A \times B) \\
\mathsf{fst} & : \mathsf{Tm}\,(A \times B) \to \mathsf{Tm}\,A \\
\mathsf{snd} & : \mathsf{Tm}\,(A \times B) \to \mathsf{Tm}\,B \\
\times\beta_1 & : \mathsf{fst}\,(a, b) = a \\
\times\beta_2 & : \mathsf{snd}\,(a, b) = b \\
\times\eta & : t = (\mathsf{fst}\,t, \mathsf{snd}\,t)
\end{array}
$$

Exercise 92. *Group the above operators into constructors and destructors. Check that the β and η rules adhere to their specification.*

Típus	Konstruktor	Destruktor	β-szabály	η-szabály
⊤ (top)	tt	nincs	nincs	t = tt
⊥ (bottom)	nincs	exfalso	nincs	f = exfalso
A + B	inl, inr	case	case f g (inl a) = f a, case f g (inr b) = g b	t = case (λa. t(inl a)) (λb. t (inr b))
A × B	(– , –)	fst, snd	fst(a, b) = a snd(a, b) = b	t = (fst t, snd t)
A ⇒ B	lam	(– · –)	lam b · a = b a	f = lam λx. f · x

β- (redukáló / számító) szabályok: mi történik, amikor azonnal használni kezded, amit építettél amikor destruktort alkalmaztál egy konstruktorra. Itt helyesek, hiszen leírják ezt a számítást, nincs kétértelműség.

η- (egyediség / kiterjeszthetőségi) szabályok: csak egy út van egy-egy adott dolog elkészítésére. A konstruktorok és a destruktorok izomorfizmust képeznek, mindegyik elem a konstruktorából. jön:

- ⊤η: egy ⊤ elem nem jöhet máshonnan, csak tt-ből (ami ⊤ egyetlen eleme)
- ⊥η: egy függvény ⊥-ból exfalso lesz (hiszen ⊥-nak nincs eleme, amit paraméterül kaphatna)
- +η: minden A + B-beli függvényt egyedien meghatározza, hogy mit tesz inl a és inr b esetén
- ×η: minden párt meghatározza a két projekciója

Exercise 93. *Define* Bool *by* $\top + \top$ *and show that the following operations are definable and equations are provable:*

$$\begin{aligned}
&\mathsf{true} && : \mathsf{Tm}\ \mathsf{Bool}\\
&\mathsf{false} && : \mathsf{Tm}\ \mathsf{Bool}\\
&\mathsf{ite} && : \mathsf{Tm}\ A \to \mathsf{Tm}\ A \to \mathsf{Tm}\ \mathsf{Bool} \to \mathsf{Tm}\ A\\
&\mathsf{Bool}\beta_1 && : \mathsf{ite}\ a\ a'\ \mathsf{true} = a\\
&\mathsf{Bool}\beta_2 && : \mathsf{ite}\ a\ a'\ \mathsf{false} = a'\\
&\mathsf{Bool}\eta && : (t : \mathsf{Tm}\ \mathsf{Bool} \to \mathsf{Tm}\ A) \to t = \mathsf{ite}\ (t\ \mathsf{true})\ (t\ \mathsf{false})
\end{aligned}$$

true := inl tt : Tm Bool
false := inr tt : Tm Bool
ite (a : Tm A) (a' : Tm A) (b : Tm Bool) := case (λ_. a) (λ_. a') b
Levezetések:

ite a a' true = (definíció)
case (λ_. a) (λ_. a') (inl tt) = (+β1)
(λ_. a) tt = (meta alkalmazás / meta β)
a

ite a a' false = (definíció)
case (λ_. a) (λ_. a') (inr tt) = (+β2)
(λ_. a') tt = (meta alkalmazás / meta β)
a'

Boolη-re alkalmazhatjuk a +η szabályt, hogy ezt kapjuk:
t = case (λa. t (inl a)) (λb. t (inr b))

Mivel a, b : Tm ⊤, a ⊤η szabály szerint a = tt és b = tt, tehát:
t (inl a) = t (inl tt) = t true
t (inr b) = t (inr tt) = t false

Tehát: t = case (λ_. t true) (λ_. t false) = lam λx. ite x (t true) (t false) = ite (t true) (t false)

Exercise 94. *Show that the terms* $\mathsf{lam}\ \lambda x.\mathsf{ite}\ x\ \mathsf{false}\ \mathsf{true}$ *and* $\mathsf{lam}\ \lambda x.(\mathsf{ite}\ x\ \mathsf{false}\ (\mathsf{ite}\ x\ \mathsf{true}\ \mathsf{true}))$ *are equal (you will need* $\mathsf{Bool}\eta$*).*

Legyen f := lam λx. ite x false (ite x true true) : Tm (Bool ⇒ Bool)

f · true = (definíció)
ite true false (ite true true true) = (⇒β, Bool β1)
false

f · false = (definíció)
ite false false (ite false true true) = (⇒β, Bool β2)
ite false true true = (Bool β2)
false

A Boolη szabályt alkalmazva a lam λx. f · x:
f = lam λx. ite x (f · true) (f · false) = lam λx. ite x false true

Exercise 95. *Define four terms of type* Bool ⇒ Bool *such that if any two are equal, then* true = false*!*

1. c_t := lam λ_. true – konstans true
2. c_f := lam λ_. false – konstans false
3. id_B := lam λx. x – identitás
4. not_B := lam λx. ite false true x – negáció

Elég könnyű belátni, hogy akármelyik kettő, ha egyenlő, akkor true = false is teljesülni fog. Lehet mindkettőre a true-t, vagy false-t alkalmazni · módon és megnézni, hogy mi lesz az eredmény.

Exercise 96. *Show that every term of type* Bool ⇒ Bool *is equal to one of the four terms defined in the previous exercise.*

Egy Bool lehet true, vagy false. Tehát Bool ⇒ Bool esetében 4 különböző érték van: a bemenet és a kimenet is lehet true / false, tehát az összes lehetőség száma 2*2=4, vagy 2^2=4.

Legyen f := lam λx. ite x (f · true) (f · false), valamint a := f · true : Tm Bool és b := f · false : Tm Bool

a = f · true	b = f · false	f = lam λx. ite x a b
true	true	c_t
true	false	id_B
false	true	not_B
false	false	c_f

Bool β szabályokkal esetlegesen belátható.

Exercise 97. *Define the three element type with a case operator and use it to define modulo 3 addition.*

Three := ⊤ + (⊤ + ⊤)

zero_Three := inl tt : Tm Three

one_Three := inr (inl tt) : Tm Three

two_Three := inr (inr tt) : Tm Three

case_Three (a b c : Tm A) (t : Tm Three) : Tm A := case (λ_. a) (λu. case (λ_. b) (λ_. c) u) t

sMod_Three := lam λn. case_Three one two zero n

addMod_Three := lam λm. lam λn. case_Three n (sMod_Three · n) (sMod_Three · (sMod_Three · n)) m

Exercise 98. *Using* Bool *and product types, define homogeneous binary sums (that is,* $A+A$ *for any* A*). Define its constructors, destructor and derive its* β *and* η *rules.*

Exercise 99. *Define the option (maybe) type former which adds an extra element to an existing type.*

Exercise 100. *For every* $n : \mathbb{N}$*, define the* Fin n *type which contains exactly* n *elements. Define its constructors, destructor.*

Exercise 101. *For every* n*, define* n*-ary product and sum types.*

Exercise 102. *Derive* $h\,(\mathsf{case}\ f\ g\ w) = \mathsf{case}\ (\lambda a.\, h\,(f\ a))\ (\lambda b.\, h\,(g\ b))\ w$ *(this is called naturality of* case *in* C*).*

Exercise 103. *Show that there is exactly one term of the following types:* $A \Rightarrow \top$ *and* $\bot \Rightarrow A$ *(for arbitrary* A*).*

Definition 104 (Type isomorphism)**.** *A and B types are isomorphic, if there are $t : \mathsf{Tm}\, A \to \mathsf{Tm}\, B$ and $t' : \mathsf{Tm}\, B \to \mathsf{Tm}\, A$, such that $t'\ (t\ a) = a$ $t\ (t'\ b) = b$ for all a, b. We denote this by $A \cong B$.*

Exercise 105. *Show $(A \times \top) \cong A$ for every A.*

Exercise 106. *Show that for any A, B we have $A + B \cong B + A$.*

The next exercise is the type theory version of the algebraic equation $(a+b)^2 = a^2 + 2ab + b^2$.

Exercise 107. *Show that for arbitrary A, B types, $\mathsf{Bool} \Rightarrow (A + B) \cong \mathsf{Bool} \Rightarrow A + \mathsf{Bool} \times A \times B + \mathsf{Bool} \Rightarrow B$.*

Exercise 108. *Show that the $\cong$ relation is reflexive, symmetric and transitive (equivalence relation), and it is a congruence for $\times$ and $+$. (For example $A \cong A'$ implies $A \times B \cong A' \times B$.)*

Definition 109 (Commutative exponential semiring)**.** *A commutative exponential semiring (commutative exponential ring without negation, commutative exponential rig) is a $\mathsf{C} : \mathsf{Set}$ with the following operations and equations. As $+$ and $*$ are commutative, we only list one identity and distributivity law. We don't give names to the equations.*

$$
\begin{aligned}
&- + - : \mathsf{C} \to \mathsf{C} \to \mathsf{C}\\
&0 \quad\ : \mathsf{C}\\
&- * - : \mathsf{C} \to \mathsf{C} \to \mathsf{C}\\
&1 \quad\ : \mathsf{C}\\
&-^{-} \quad : \mathsf{C} \to \mathsf{C} \to \mathsf{C}\\
&\quad\ : (a + b) + c = a + (b + c)\\
&\quad\ : a + b = b + a\\
&\quad\ : 0 + a = a\\
&\quad\ : (a * b) * c = a * (b * c)\\
&\quad\ : a * b = b * a\\
&\quad\ : 1 * a = a\\
&\quad\ : a * (b + c) = a * b + a * c\\
&\quad\ : a^{b+c} = a^b * a^c\\
&\quad\ : a^0 = 1\\
&\quad\ : (a * b)^c = a^c * b^c\\
&\quad\ : 1^c = 1\\
&\quad\ : a^{b*c} = (a^b)^c\\
&\quad\ : a^1 = a
\end{aligned}
$$

Exercise 110. *(Meta) natural numbers form a commutative exponential semiring with addition, multiplication and exponentiation. They also form a commutative semiring with maximum and addition. Natural numbers extended with infinity form a commutative semiring with minimum and addition.*

Exercise 111. *In the language Fin, types form a commutative semiring where equations are isomorphisms.*

Remark 112. *Commutative exponential semirings are what Tarski calls high school algebra [BL93] (although they omit the operation 0). A related interesting completeness problem was settled by Wilkie [Wil01]. The dependently typed analogue is still open [Alt08].*

Section 3.3 takes the STLC (functions) from Section 3.1 and adds four new "shapes" of type.

⊤ (Unit) — "The One-Thing Box" A type containing exactly one element: tt. You never have to make a choice; there's only one thing to put in this box. It's useful as a "placeholder" or a "done" signal. If you have *anything* of type ⊤, it *must* be tt — the η rule just says "there's only one".

⊥ (Empty/Bottom) — "The Impossible Box" A type with *zero* elements. You can never actually build something of type ⊥. But *if* you somehow had one (which is impossible!), you could magically produce anything at all. That's exfalso — "from a false assumption, anything follows." This is just classical logic: a proof of False lets you prove everything.

A + B (Sum/Coproduct) — "The Either Box" An element is *either* something from A *or* something from B, and there's a **tag** saying which. Like a sealed envelope that contains either a letter or a photo, and there's a sticker saying which one.

- inl a = "I chose the left, here's an A"
- inr b = "I chose the right, here's a B"
- case f g w = "Open the envelope and handle both possibilities"

The η rule says: any function *out of* A+B must be a case — there's no other way to handle both possibilities.

A × B (Product) — "The Pair Box" An element is *both* an A *and* a B simultaneously. Like a box with two compartments.

- (a, b) = "Here's the pair"
- fst = "Give me the first component"
- snd = "Give me the second component"

The η rule says any element of A×B is fully determined by its two components: w = (fst w, snd w).

The Deep Pattern (β and η rules)

Every type follows the same rhythm:

Type	Constructor (builds it)	Destructor (uses it)	β rule	η rule
⊤	tt	(none)	—	t = tt
⊥	(none)	exfalso	—	f = exfalso
A+B	inl, inr	case	case f g (inl a) = f a	t = case (λa. t(inl a)) …
A×B	(–,–)	fst, snd	fst(a,b) = a	t = (fst t, snd t)

β rules = "what happens when you immediately open something you just built" (reduce/compute). **η rules** = "there's only one way to make this kind of thing" (uniqueness/extensionality).

Why is this section called "Finite Types"? Because combining ⊥, ⊤, +, and × lets you build any type with a *finite* number of elements. ⊥ has 0, ⊤ has 1, Bool = ⊤+⊤ has 2, Three = ⊤+⊤+⊤ has 3, etc. Adding function types (⇒) gives infinite types.

3.4 Generic programming

In this section we follow the book of Robert Harper [Har16].

Assume we have a Bool ⇒ Nat function, one which maps true to 1 and false to 0 (e.g. lam (ite (suc zero) zero)). We can apply this to a term of type Bool. But we can also apply it to a term in Bool × Nat in a way that the Nat part is not touched, i.e. (t, t') is mapped to (ite (suc zero) zero t, t'). Or we can apply it to a term in (Nat + Bool) × Bool where we apply it to the two Bools and we leave the Nat.

In general: we want to extend an $A \Rightarrow B$ function to an $F : \mathsf{Ty} \to \mathsf{Ty}$ type operator (a type depending on a type) in such a way that we obtain an $F\,A \Rightarrow F\,B$ function. This is what generic programming allows us for certain F type operators. An $F = \lambda X.\,A$, where A can only contain X, nullary and binary (co)products is called a polynomial type operator. If A contains function types, then the positive type operators are those for which the function can be extended to.

The type operator corresponding to the polynomial $\lambda x.2x^3 + x^2 + 3x + 1$ is $\lambda X.(\top + \top) \times (X \times X \times X) + X \times X + (\top + \top + \top) \times X + \top$.

Definition 113 (Map for polynomial type operators)**.** *For a second-order model of Fin (Definition 91), a model of map for polynomial type operators has one sort* Poly*, and some unnamed operators:*

$$\begin{aligned}
\mathsf{Poly} &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set} \\
&: \mathsf{Poly}\,(\lambda X.X) \\
&: \mathsf{Poly}\,(\lambda X.\top) \\
&: \mathsf{Poly}\,F \to \mathsf{Poly}\,G \to \mathsf{Poly}\,(\lambda X.F\,X \times G\,X) \\
&: \mathsf{Poly}\,(\lambda X.\bot) \\
&: \mathsf{Poly}\,F \to \mathsf{Poly}\,G \to \mathsf{Poly}\,(\lambda X.F\,X + G\,X)
\end{aligned}$$

There is also an operator map *with some unnamed equations:*

$$\begin{aligned}
&\mathsf{map} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{Poly}\,F\} \to (\mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,(F\,B) \\
&\mathsf{map}\,(\lambda X.X)\quad\quad f\,a = f\,a \\
&\mathsf{map}\,(\lambda X.\top)\quad\quad f\,t = \mathsf{tt} \\
&\mathsf{map}\,(\lambda X.F\,X \times G\,X)\;f\,w = (\mathsf{map}\,F\,f\,(\mathsf{fst}\,w), \mathsf{map}\,G\,f\,(\mathsf{snd}\,w)) \\
&\mathsf{map}\,(\lambda X.\bot)\quad\quad f\,t = \mathsf{exfalso}\,t \\
&\mathsf{map}\,(\lambda X.F\,X + G\,X)\;f\,w = \mathsf{case}\,w\,(\lambda u.\mathsf{inl}\,(\mathsf{map}\,F\,f\,u))\,(\lambda v.\mathsf{inr}\,(\mathsf{map}\,G\,f\,v))
\end{aligned}$$

Exercise 114. *Show that the following type operator is polynomial.*

$$\lambda X.\bot + (\top \times X + X \times X)$$

$$\dfrac{\dfrac{\dfrac{}{\mathsf{Poly}\,(\lambda X.\,\bot)}\,[\mathrm{bottom}] \quad \dfrac{\dfrac{}{\mathsf{Poly}\,(\lambda X.\,\top)}\,[\mathrm{top}] \quad \dfrac{}{\mathsf{Poly}\,(\lambda X.\,X)}\,[\mathrm{id}]}{\mathsf{Poly}\,(\lambda X.\,\top \times X)}\,[\mathrm{prod}] \quad \dfrac{\dfrac{}{\mathsf{Poly}\,(\lambda X.\,X)}\,[\mathrm{id}] \quad \dfrac{}{\mathsf{Poly}\,(\lambda X.\,X)}\,[\mathrm{id}]}{\mathsf{Poly}\,(\lambda X.\,X \times X)}\,[\mathrm{prod}]}{\mathsf{Poly}\,(\lambda X.\,\top \times X + X \times X)}\,[\mathrm{sum}]}{\mathsf{Poly}\,(\lambda X.\,\bot + (\top \times X + X \times X))}\,[\mathrm{sum}]$$

Exercise 115. *Derive* $\mathsf{map}\,(\lambda X.\top + (\mathsf{Bool} \times X))\,f\,(\mathsf{inr}\,(\mathsf{true}, t)) = \mathsf{inr}\,(\mathsf{true}, f\,t)$.

Először a +-ra, majd a ×-ra kell map-et alkalmazni, majd kiértékelni mindegyiket – így levezethető:

map (λX. ⊤ + (Bool × X)) f (inr (true, t)) = (map a +-hoz)
case (inr (true, t)) (λu. inl (map (λX. ⊤) f u)) (λv. inr (map (λX. Bool × X) f v)) = (+β2)
inr (map (λX. Bool × X) f (true, t)) = (map a ×-hoz)
inr (map (λX. Bool) f (fst (true, t)), map (λX. X) f (snd (true, t))) = (×β1, ×β2)
inr (map (λX. Bool) f true, map (λX. X) f t) = (kétfelé bontás jön)

map (λC. Bool) f true = case true (λu. inl (map (λX. ⊤) f u)) (λv. inr (map (λX. ⊤) f v)) (map a +-hoz, hiszen Bool = ⊤ + ⊤) = (+β1: true = inl tt)
inl (map (λX. ⊤) f tt) = (map a ⊤-hoz: tt-ad vissza)
true

map (λX. X) f t = f t (map az identitáshoz)

= inr (true, f t)

Exercise 116. *Assuming* Poly *is the initial* Poly *model (that is, a meta inductive type), define* map *as iteration into a* Poly *model.*

Exercise 117. *Assuming* map *is defined as in Exercise 123, show that if* $A : \mathsf{Ty}$*, then* $\mathsf{map}\,(\lambda X.A)\,f\,w = w$.

Exercise 118. *Assuming* map *is defined as in Exercise 123, show that it is a functor:* $\mathsf{map}\,F\,(\lambda x.f\,(g\,x))\,w = \mathsf{map}\,F\,f\,(\mathsf{map}\,F\,g\,w)$ *and* $\mathsf{map}\,F\,(\lambda x.x)\,u = u$.

If we also allow functions in the type operator, then sometimes we cannot extend our type operator. For example there is no map function for the type operators

- $F\,X = X \Rightarrow \bot$,
- $F\,X = X \Rightarrow X$,

however there is a map function for

- $F\,X = \mathsf{Bool} \Rightarrow X$,
- $F\,X = ((X + X) \Rightarrow \mathsf{Bool}) \Rightarrow X$.

In the following language we introduce the notions of positive and negative type operators. We have a (covariant) map for the positive type operators while we have a contravariant map for the negative ones.

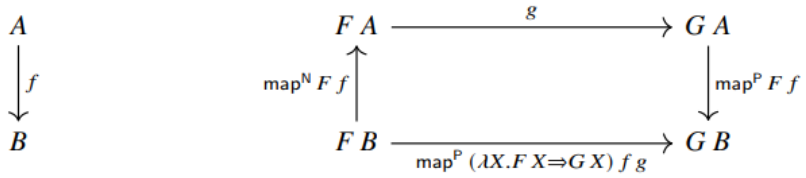
Definition 119 (Map for positive type operators)**.** *For a second-order model of Fin (Definition 91), a model of map for positive type operators has two sorts* Pos *and* Neg *and some unnamed operators:*

$$
\begin{aligned}
&\mathsf{Pos} : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set}\\
&\mathsf{Neg} : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set}\\
&\quad : \mathsf{Pos}\,(\lambda X.X)\\
&\quad : (C : \mathsf{Ty}) \to \mathsf{Pos}\,(\lambda_.C)\\
&\quad : (C : \mathsf{Ty}) \to \mathsf{Neg}\,(\lambda_.C)\\
&\quad : \mathsf{Pos}\,F \to \mathsf{Pos}\,G \to \mathsf{Pos}\,(\lambda X.F\,X + G\,X)\\
&\quad : \mathsf{Neg}\,F \to \mathsf{Neg}\,G \to \mathsf{Neg}\,(\lambda X.F\,X + G\,X)\\
&\quad : \mathsf{Pos}\,F \to \mathsf{Pos}\,G \to \mathsf{Pos}\,(\lambda X.F\,X \times G\,X)\\
&\quad : \mathsf{Neg}\,F \to \mathsf{Neg}\,G \to \mathsf{Neg}\,(\lambda X.F\,X \times G\,X)\\
&\quad : \mathsf{Neg}\,F \to \mathsf{Pos}\,G \to \mathsf{Pos}\,(\lambda X.F\,X \Rightarrow G\,X)\\
&\quad : \mathsf{Pos}\,F \to \mathsf{Neg}\,G \to \mathsf{Neg}\,(\lambda X.F\,X \Rightarrow G\,X)
\end{aligned}
$$

There are also two operators satisfying the equations below:

$$\begin{aligned}
&\mathsf{map}^{\mathsf{P}} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{Pos}\, F\} \to (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \to \mathsf{Tm}\,(F\, A) \to \mathsf{Tm}\,(F\, B)\\
&\mathsf{map}^{\mathsf{N}} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{Neg}\, F\} \to (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \to \mathsf{Tm}\,(F\, B) \to \mathsf{Tm}\,(F\, A)\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.X) \qquad f\ a = f\ a\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.C) \qquad f\ c = c\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.C) \qquad f\ c = c\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.F\ X + G\ X) \quad f\ w = \mathsf{case}\ w\ (\lambda u.\mathsf{inl}\ (\mathsf{map}^{\mathsf{P}}\ F\ f\ u))\ (\lambda v.\mathsf{inr}\ (\mathsf{map}^{\mathsf{P}}\ G\ f\ v))\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.F\ X + G\ X) \quad f\ w = \mathsf{case}\ w\ (\lambda u.\mathsf{inl}\ (\mathsf{map}^{\mathsf{N}}\ F\ f\ u))\ (\lambda v.\mathsf{inr}\ (\mathsf{map}^{\mathsf{N}}\ G\ f\ v))\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.F\ X \times G\ X) \quad f\ w = (\mathsf{map}^{\mathsf{P}}\ F\ f\ (\mathsf{fst}\ w), \mathsf{map}^{\mathsf{P}}\ G\ f\ (\mathsf{snd}\ w))\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.F\ X \times G\ X) \quad f\ w = (\mathsf{map}^{\mathsf{N}}\ F\ f\ (\mathsf{fst}\ w), \mathsf{map}^{\mathsf{N}}\ G\ f\ (\mathsf{snd}\ w))\\
&\mathsf{map}^{\mathsf{P}}\ (\lambda X.F\ X \Rightarrow G\ X)\ f\ g = \mathsf{lam}\ \lambda x.\mathsf{map}^{\mathsf{P}}\ G\ f\ (g \cdot (\mathsf{map}^{\mathsf{N}}\ F\ f\ x))\\
&\mathsf{map}^{\mathsf{N}}\ (\lambda X.F\ X \Rightarrow G\ X)\ f\ g = \mathsf{lam}\ \lambda x.\mathsf{map}^{\mathsf{N}}\ G\ f\ (g \cdot (\mathsf{map}^{\mathsf{P}}\ F\ f\ x))
\end{aligned}$$

We illustrate the $\mathsf{map}^{\mathsf{P}}$-equation for $\Rightarrow$ by the following diagram.



Exercise 120. *Show that the following type operators are positive, negative or both.*

- $FX = X \Rightarrow \mathsf{Bool} \Rightarrow \mathsf{Bool}$
- $FX = (X \Rightarrow \mathsf{Bool}) \Rightarrow X$
- $FX = ((X \Rightarrow \mathsf{Bool}) \Rightarrow \mathsf{Bool}) \Rightarrow \mathsf{Bool}$
- $FX = (\mathsf{Bool} \Rightarrow \mathsf{Bool}) \Rightarrow X + X$
- $FX = (X \times (X + \top)) \Rightarrow \mathsf{Bool}$
- $FX = (\mathsf{Bool} \Rightarrow \mathit{Bool}) \Rightarrow \top$

Negatív, pozitív, negatív, pozitív, negatív, mindkettő.

Exercise 121. *Show that if the type operator $F\ X = X \Rightarrow \bot$ comes with a $\mathsf{map}^{\mathsf{P}}$ function, then $\mathsf{Tm}\,\bot$ has an element.*

Food for thought 122. *The following type operators are not positive, nor negative. What sort of contradiction do we get if they come equipped with $\mathsf{map}^{\mathsf{P}}$ or $\mathsf{map}^{\mathsf{N}}$ functions?*

- $FX = X \Rightarrow X$
- $FX = (\mathsf{Bool} \Rightarrow X) \Rightarrow X$

Exercise 123. *Assuming* Pos *and* Neg *are the initial* Pos–Neg *model (that is, two mutually defined meta inductive types), define $\mathsf{map}^{\mathsf{P}}$ and $\mathsf{map}^{\mathsf{N}}$ by iteration into a* Pos–Neg *model.*

Exercise 124. *Prove that $\mathsf{map}^{\mathsf{P}}$ is a covariant functor, while $\mathsf{map}^{\mathsf{N}}$ is contravariant:*

$$\begin{aligned}
&\mathsf{map}^{\mathsf{P}}\ F\ (\lambda x.f\ (g\ x))\ w = \mathsf{map}^{\mathsf{P}}\ F\ f\ (\mathsf{map}^{\mathsf{P}}\ F\ g\ w) \qquad &&\mathsf{map}^{\mathsf{P}}\ F\ (\lambda x.x)\ u = u\\
&\mathsf{map}^{\mathsf{N}}\ F\ (\lambda x.f\ (g\ x))\ w = \mathsf{map}^{\mathsf{N}}\ F\ g\ (\mathsf{map}^{\mathsf{N}}\ F\ f\ w) \qquad &&\mathsf{map}^{\mathsf{N}}\ F\ (\lambda x.x)\ u = u
\end{aligned}$$

The Core Problem

Suppose you have a function f that converts kilograms to pounds. You have a box of groceries listed in kg. Can you automatically apply f to *every* item in the box, keeping the box structure intact?

That is exactly what generic programming is about. Given f : Tm A → Tm B, can we automatically lift f to work on *any* type that *contains* A — things like A × C, A + C, C ⇒ A, etc. — by applying f wherever an A appears?

Polynomial Type Operators

A **type operator** F : Ty → Ty takes a type and builds a new type. For example:

- F = λX. X × Bool takes A and gives A × Bool
- F = λX. X + X takes A and gives A + A

A **polynomial** type operator is one built *only* from X, ⊤, ⊥, +, and × — no function types. Think of it as a polynomial expression in X:

Polynomial expression	Type operator	Example instance at A = Nat
x	λX. X	Nat
1	λX. ⊤	⊤
0	λX. ⊥	⊥
$x^2 + x + 3$	λX. X×X + X + ⊤+⊤+⊤	Nat×Nat + Nat + ⊤+⊤+⊤

For a polynomial operator F, the map function lifts any f : Tm A → Tm B to map F f : Tm (F A) → Tm (F B). It applies f wherever an A appears inside F A, leaving everything else alone.

How map works for each kind of operator:

- map X f a = f a — directly apply f
- map ⊤ f t = tt — nothing to change in ⊤
- map ⊥ f t = exfalso t — impossible case
- map (F×G) f (a, b) = (map F f a, map G f b) — apply to both components
- map (F+G) f w = case w (inl ∘ map F f) (inr ∘ map G f) — apply in whichever side is present

Positive and Negative Operators

When we allow **function types** (A ⇒ B), a variable X can appear in two kinds of positions:

🟢 **Positive (covariant) position**: X is in the *output* of a function. If X grows to B (f : A→B), we can map *forward*. Like a function that produces X — a bigger X is fine.

🔴 **Negative (contravariant) position**: X is in the *input* of a function. To go from A-version to B-version, you need to go *backwards* (use f "in reverse"). Like a function that *consumes* X — you can only substitute if you have a backwards map.

The counting rule (like in logic):

- Each time X crosses a function arrow going left-to-right, it flips between positive and negative
- X starts positive, X ⇒ C makes X negative, (X ⇒ C) ⇒ D makes X positive again (double flip), etc.

mapP (covariant) works for operators where X is *only positive*. mapN (contravariant) works for operators where X is *only negative*.

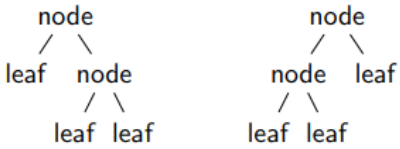
Intuition: Think of type operators like glasses lenses. A positive operator magnifies: making A bigger (A→B) makes F A bigger too. A negative operator inverts: making A bigger makes F A smaller, because F A *consumes* elements of A.

3.5 Inductive types

Bool in Razor (Definition 14) and Nat in System T (Definition 76) were *inductive types*. Another example is the type of binary trees Tree which has two constructors:

$$\begin{aligned}\mathsf{Tree} &: \mathsf{Ty}\\ \mathsf{leaf} &: \mathsf{Tm}\ \mathsf{Tree}\\ \mathsf{node} &: \mathsf{Tm}\ \mathsf{Tree} \to \mathsf{Tm}\ \mathsf{Tree} \to \mathsf{Tm}\ \mathsf{Tree}\end{aligned}$$

The trees in Tree have binary branching and there is no information at the branching nodes or at the leaves. For example, the trees node leaf (node leaf leaf) and node (node leaf leaf) leaf are depicted as follows.



A Tree algebra (also called a Tree model inside our object language) is given by the following components:

$$A : \mathsf{Ty} \qquad u : \mathsf{Tm}\, A \qquad v : \mathsf{Tm}\, A \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A,$$

A homomorphism from the algebra (Tree, leaf, node) to the algebra (A, u, v) is given by a function which respects the operations as follows:

$$\begin{aligned} &t : \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\, A \\ &t\,\mathsf{leaf} = u \\ &t\,(\mathsf{node}\, l\, r) = v\,(t\, l)\,(t\, r) \end{aligned}$$

The iterator of Tree says that for any algebra (A, u, v), there is such a homomorphism. The two equations that the homomorphism satisfies are called computation (β) rules. The uniqueness (η) rule says that every other homomorphism from (Tree, leaf, node) to (A, u, v) is equal to the iterator.

The two constructors of Tree can be merged into a single constructor along the following isomorphisms:

$$\begin{aligned} &\mathsf{Tm}\,\mathsf{Tree} \times (\mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,\mathsf{Tree}) && \cong \\ &(\mathsf{Tm}\,\top \to \mathsf{Tm}\,\mathsf{Tree}) \times (\mathsf{Tm}\,\mathsf{Tree} \times \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\,\mathsf{Tree}) && \cong \\ &\mathsf{Tm}\,(\top \Rightarrow \mathsf{Tree}) \times \mathsf{Tm}\,((\mathsf{Tree} \times \mathsf{Tree}) \Rightarrow \mathsf{Tree}) && \cong \\ &\mathsf{Tm}\left((\top \Rightarrow \mathsf{Tree}) \times ((\mathsf{Tree} \times \mathsf{Tree}) \Rightarrow \mathsf{Tree})\right) && \cong \\ &\mathsf{Tm}\,((\top + \mathsf{Tree} \times \mathsf{Tree}) \Rightarrow \mathsf{Tree}) && \cong \\ &\mathsf{Tm}\,(\top + \mathsf{Tree} \times \mathsf{Tree}) \to \mathsf{Tm}\,\mathsf{Tree} && = \\ &\mathsf{Tm}\,((\lambda X.\top + X \times X)\,\mathsf{Tree}) \to \mathsf{Tm}\,\mathsf{Tree} \end{aligned}$$

Definition 125 (Algebra of a type operator)**.** *Assume a type operator* $F : \mathsf{Ty} \to \mathsf{Ty}$*. An* F*-algebra consists of a type* $A : \mathsf{Ty}$ *and a function* $\mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,A$*.*

An inductive type consists of an F-algebra and an iterator. The type in the F-algebra is what we call the inductive type itself. The constructors of the inductive type are given by the function $\mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,A$ function in the F-algebra. The iterator encodes that there is a unique F-homomorphism from the inductive type into any other F-algebra. This is why the inductive type is also called the initial F-algebra. (It could be also called the syntax for F, but note that this syntax is internal to a model of our object theory.)

In order to obtain a well-defined notion of F-homomorphism, we need to restrict the type operator F to be positive. We will be even more restrictive, we will only allow strictly positive type operators. There are more strictly positive type operators than polynomial type operators, but less than positive type operators.

Food for thought 126. *Assuming that positive type operators have initial algebras is inconsistent with excluded middle.*

The Tree type is determined by the $\lambda X.\top + X \times X$ type operator, its constructor is

$$\mathsf{con} : \mathsf{Tm}\,(\top + \mathsf{Tree} \times \mathsf{Tree}) \to \mathsf{Tm}\,\mathsf{Tree},$$

its iterator and its computation rule are the following:

$$\begin{aligned} &\mathsf{ite} : \left(\mathsf{Tm}\,(\top + A \times A) \to \mathsf{Tm}\, A\right) \to \mathsf{Tm}\,\mathsf{Tree} \to \mathsf{Tm}\, A \\ &\mathsf{ite}\, f\,(\mathsf{con}\, w) = \mathsf{case}\, w\,(\lambda_.f\,(\mathsf{inl}\,\mathsf{tt}))\left(\lambda u.f\left(\mathsf{inr}\left(\mathsf{ite}\, f\,(\mathsf{fst}\, u), \mathsf{ite}\, f\,(\mathsf{snd}\, u)\right)\right)\right) \end{aligned}$$

The following language contains an inductive type for all F strictly positive type operators. We express the equation for the homomorphism using map.

Definition 127 (Language with inductive types)**.** *A second-order model extends Fin (Definition 91) with the following components:*

$$\begin{aligned}
\mathsf{SPos} &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Set} \\
&: \mathsf{SPos}\,(\lambda X.X) \\
&: (A : \mathsf{Ty}) \to \mathsf{SPos}\,(\lambda _.A) \\
&: \mathsf{SPos}\,F \to \mathsf{SPos}\,G \to \mathsf{SPos}\,(\lambda X.F\,X + G\,X) \\
&: \mathsf{SPos}\,F \to \mathsf{SPos}\,G \to \mathsf{SPos}\,(\lambda X.F\,X \times G\,X) \\
&: (C : \mathsf{Ty}) \to \mathsf{SPos}\,F \to \mathsf{SPos}\,(\lambda X.C \Rightarrow F\,X)
\end{aligned}$$

We also have a map *operator:*

$$\begin{aligned}
&\mathsf{map} : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,F\} \to (f : \mathsf{Tm}\,A \to \mathsf{Tm}\,B) \to \mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,(F\,B) \\
&\mathsf{map}\,(\lambda X.X) \qquad f\,a = f\,a \\
&\mathsf{map}\,(\lambda _.C) \qquad f\,c = c \\
&\mathsf{map}\,(\lambda X.F\,X + G\,X)\ f\,w = \mathsf{case}\,w\,(\lambda u.\mathsf{inl}\,(\mathsf{map}\,F\,f\,u))\,(\lambda v.\mathsf{inr}\,(\mathsf{map}\,G\,f\,v)) \\
&\mathsf{map}\,(\lambda X.F\,X \times G\,X)\ f\,w = \big(\mathsf{map}\,F\,f\,(\mathsf{fst}\,w), \mathsf{map}\,G\,f\,(\mathsf{snd}\,w)\big) \\
&\mathsf{map}\,(\lambda X.C \Rightarrow F\,X)\ \ f\,g = \mathsf{lam}\,\lambda a.\mathsf{map}\,F\,f\,(g \cdot a)
\end{aligned}$$

And inductive types with their constructor, iterator, computation and uniqueness rules:

$$\begin{aligned}
&\mathsf{Ind} \quad : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \mathsf{Ty} \\
&\mathsf{con} \quad : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \mathsf{Tm}\,\big(F\,(\mathsf{Ind}\,F)\big) \to \mathsf{Tm}\,(\mathsf{Ind}\,F) \\
&\mathsf{ite} \quad : (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \big(\mathsf{Tm}\,(F\,A) \to \mathsf{Tm}\,A\big) \to \mathsf{Tm}\,(\mathsf{Ind}\,F) \to \mathsf{Tm}\,A \\
&\mathsf{Ind}\beta : \mathsf{ite}\,F\,f\,(\mathsf{con}\,F\,x) = f\,\big(\mathsf{map}\,F\,(\mathsf{ite}\,F\,f)\,x\big) \\
&\mathsf{Ind}\eta : (z : \mathsf{Tm}\,(\mathsf{Ind}\,F) \to \mathsf{Tm}\,A) \to \Big(\big(\lambda x.z\,(\mathsf{con}\,F\,x)\big) = \big(\lambda x.f\,(\mathsf{map}\,F\,z\,x)\big)\Big) \to z = \mathsf{ite}\,F\,f
\end{aligned}$$

Remark 128. *Usually η rules are omitted because (i) they are not needed for running programs; (ii) equality checking in the syntax becomes undecidable in the presence of η for natural numbers. This is not the case for η for finite types [Sch17].*

Inductive types are trees with finite depth. This means that they might have infinitely many branches, but if we follow a branch, then in finite steps we reach a leaf. Note that this does not mean that we can put an element of an inductive type into a hole of finite depth: there are infinitely branching trees where every branch is finite but it has branches of arbitrary length.

Exercise 129. *The inductive type* $\mathsf{Ind}\,(\lambda X.\top + (\mathsf{Nat} \Rightarrow X))$ *contains infinitely branching trees. Define a tree in which the ith branch has height i.*

T := Ind (λX. ⊤ + (Nat ⇒ X))

leaf : Tm T := con (inl tt)

node := Tm (Nat ⇒ T) → Tm T := lam λf. con (inr f)

Exercise 130. *Natural numbers are given by* $\mathsf{Nat} := \mathsf{Ind}\,(\lambda X.\top + X)$*. Define* zero *and* suc *using* con*, define the iterator and prove the computation rules.*

zero : Tm Nat := con (inl tt)

suc : Tm Nat → Tm Nat := lam λn. con (inr n)

iteNat z s n := ite (case (λ_. z) s) n

Exercise 131. *Define the following inductive types: lists of natural numbers, binary trees with natural numbers at the leaves, binary trees with natural numbers at the nodes, ternary trees, untyped combinator calculus (without equations), Razor (without equations or types).*

Természetes számok: λX. ⊤ + (Nat × X)
Bináris fa, leveleknél Nat: λX. Nat + (X × X)
Bináris fa, csúcsoknál Nat: λX. ⊤ + (Nat × X × X)
Ternáris fa: λX. ⊤ + X × X × X
Típusozatlan kombinátor kalkulus (egyenletek nélkül): λX. ⊤ + ⊤ + X × X
Razor (-egyenletek, -típusok): λX. Nat + X × X + X + ⊤ + ⊤ + X × X × X (num plus, isZero, true, false, ite)

Exercise 132. $\mathsf{Ind}\,F$ *is the fixpoint of the* F *function:* $F\,(\mathsf{Ind}\,F) \cong \mathsf{Ind}\,F$.

Exercise 133. *Show that for every* A*, we have* $\mathsf{Ind}\,(\lambda X.A) \cong A$.

The recursor of Nat can be defined via the iterator: we use the iterator in a way that it not only returns A, but also the concrete natural number on which it was called. For reference, we also list the rules for the iterator:

$$
\begin{aligned}
&\mathsf{ite} \quad : \{A : \mathsf{Ty}\} \to \mathsf{Tm}\,A \to (\mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A\\
&\mathsf{ite}\beta_1 : \mathsf{ite}\,z\,s\,\mathsf{zero} = z\\
&\mathsf{ite}\beta_2 : \mathsf{ite}\,z\,s\,(\mathsf{suc}\,n) = s\,(\mathsf{ite}\,z\,s\,n)\\
&\mathsf{ite}\eta \quad : (z : \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A) \to (z\,\mathsf{zero} = z) \to ((n : \mathsf{Tm}\,\mathsf{Nat}) \to z\,(\mathsf{suc}\,n) = s\,(z\,n)) \to z = \mathsf{ite}\,z\,s\\
&\mathsf{rec} \quad : \{A : \mathsf{Ty}\} \to \mathsf{Tm}\,A \to (\mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,\mathsf{Nat} \to \mathsf{Tm}\,A\\
&\mathsf{rec}\,\{A\}\,z\,s\,n := \mathsf{snd}\left(\mathsf{ite}\{\mathsf{Nat} \times A\}\,(\mathsf{zero}, z)\,(\lambda w.(\mathsf{suc}\,(\mathsf{fst}\,w), s\,(\mathsf{fst}\,w)\,(\mathsf{snd}\,w)))\,n\right)
\end{aligned}
$$

Exercise 134. *Prove the* β *rules of the recursor:*

$$
\begin{aligned}
&\mathsf{rec}\beta_1 : \mathsf{rec}\,z\,s\,\mathsf{zero} = z\\
&\mathsf{rec}\beta_2 : \mathsf{rec}\,z\,s\,(\mathsf{suc}\,n) = s\,n\,(\mathsf{rec}\,z\,s\,n)
\end{aligned}
$$

For the latter we need the η *law of the iterator (without this,* $\mathsf{rec}\beta_2$ *is not derivable [Par89]).*

Food for thought 135. *Define the* η *rule of the recursor. Can it be derived from that of the iterator?*

Food for thought 136. *Define the recursor for an arbitrary inductive type and show that it can be derived from the iterator.*

If we only have inductive types for polynomial type operators, we obtain finitely branching trees.

Food for thought 137. *For every* F *polynomial type operator,* $\mathsf{Ind}\,F$ *is either isomorphic to a finite type or isomorphic to* Nat.

Exercise 138 (Cantor's diagonal argument)**.** *Show that if* $(\mathsf{Nat} \Rightarrow \mathsf{Bool}) \cong \mathsf{Nat}$*, then* $\mathsf{true} = \mathsf{false}$.

The Core Idea: Trees All the Way Down

Every inductive type is a type of **trees**. The functor F tells you the "shape" of each node — how many children it can have, what extra data it holds. Ind F is then the type of all possible trees following that shape. Think of F as a cookie cutter. The cutter's shape describes *one layer* of the tree. Ind F is the type of all cookies you can make by stacking this cutter infinitely many times.

F-algebras: "What do I do when I see this shape?"

An **F-algebra** is a type A together with a function f : Tm(F A) → Tm A. It answers: *"if I see an F-shaped bundle of A-values, what A-value do I produce?"*

For the Tree type (F = λX. ⊤ + X × X):

- An algebra says: "given either nothing (leaf) or two A-values (left/right subtree), give me an A"
- Example algebras: counting nodes, computing height, collecting leaves into a list.

Ind F is the most "honest" algebra — it doesn't compute anything. It just stores the tree structure. It's the **initial** algebra: the most abstract one from which every other algebra receives a unique map.

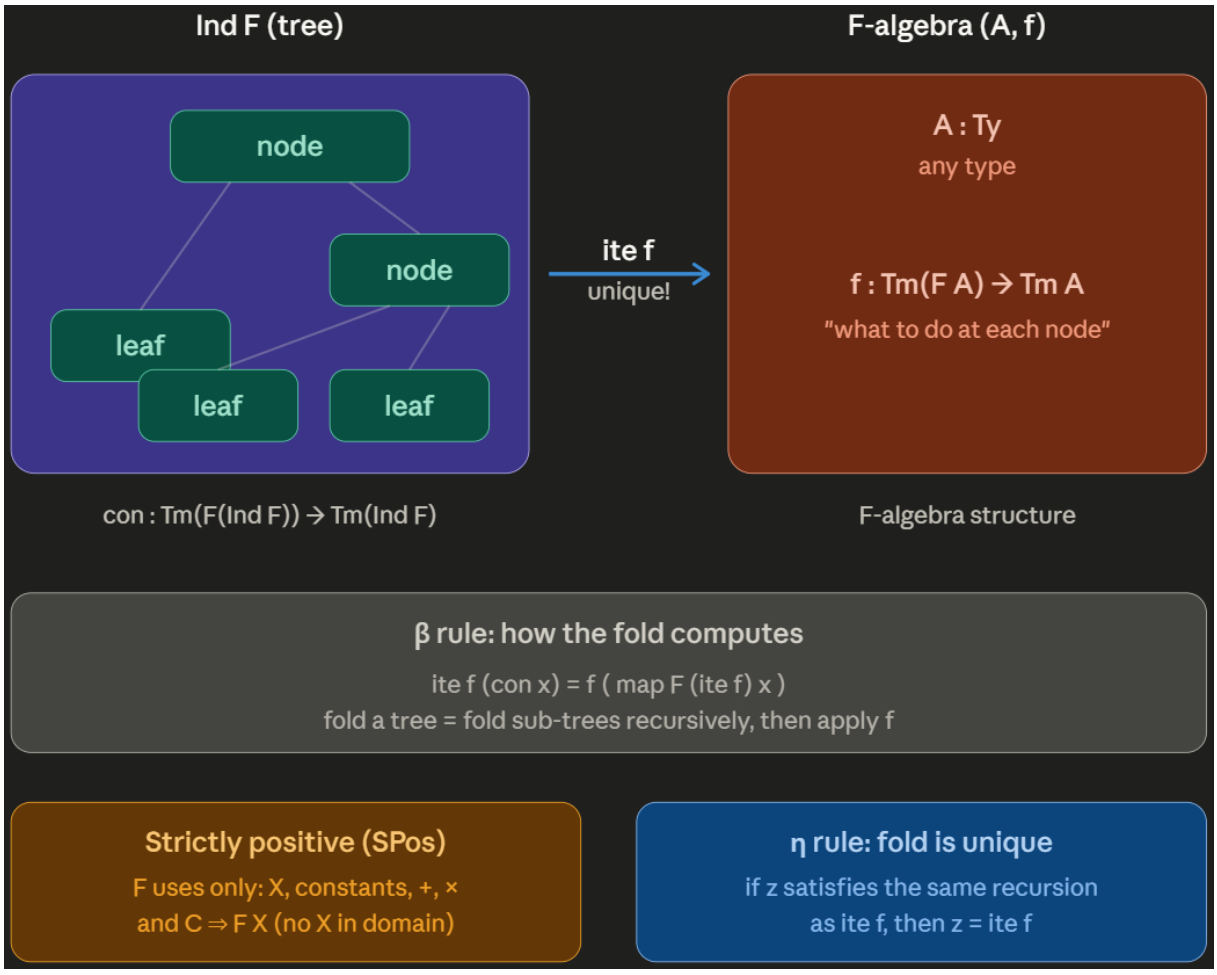
The Iterator: The Universal Fold

ite f t folds tree t using algebra f. The β rule says exactly what this means:

ite f (con x) = f (map F (ite f) x)

In plain English: *"to fold a tree, first recursively fold all the sub-trees, then apply f to the resulting bundle."* This is the **catamorphism** / **fold** pattern. It's the recursion scheme for inductive types.

Let me show this with a diagram, here's how the iterator works on a binary tree:



Why "Strictly Positive"?

The SPos restriction bans X from appearing in the *domain* of a function type inside F. Why? Because F X = X ⇒ ⊥ would give an inductive type that contains functions *eating elements of itself* — this leads to paradoxes (like Girard's paradox or Russell's paradox). Strictly positive operators only let X appear in covariant positions (outputs), guaranteeing a well-founded tree structure.

The recursor vs the iterator: The iterator ite's step function only sees the *result* of the recursive calls. The recursor rec's step function also sees the *original value* (the current nat n in rec's case). The recursor is stronger — it's needed to define predecessor — but it can always be derived from the iterator using a "pairing trick": iterate over pairs (current_value, accumulated_result).

3.6 Coinductive types

Coinductive types are dual to inductive types.

- Inductive types are determined by their constructors, coinductive types by their destructors.
- The functions which we define out of inductive types using the iterator always *terminate*, the generators creating elements of coinductive types are always *productive*.
- Elements of inductive types are trees with finite depth, while coinductive types generate trees of potentially infinite depth.

Programs running forever (e.g. servers, operating systems, interactive programs), automata, Turing-machines and other machines are elements of coinductive types. They also model parts of object oriented programmign: a class is a coinductive type, the fields/member functions are the destructors, an object is an element of the coinductive type. In the 1990s Turner suggested strong functional programming [**?**] as a style where we separate inductive and coinductive types and there is no general recursion. No one has ever tried programming in such a setting.

The most well-known example of a coinductive type is the type of streams (lists of infinite length).

$$
\begin{array}{ll}
\mathsf{Stream} & : \mathsf{Ty} \to \mathsf{Ty} \\
\mathsf{head} & : \mathsf{Tm}\,(\mathsf{Stream}\,A) \to \mathsf{Tm}\,A \\
\mathsf{tail} & : \mathsf{Tm}\,(\mathsf{Stream}\,A) \to \mathsf{Tm}\,(\mathsf{Stream}\,A) \\
\mathsf{gen} & : (\mathsf{Tm}\,C \to \mathsf{Tm}\,A) \to (\mathsf{Tm}\,C \to \mathsf{Tm}\,C) \to \mathsf{Tm}\,C \to \mathsf{Tm}\,(\mathsf{Stream}\,A) \\
\mathsf{Stream}\beta_1 & : \mathsf{head}\,(\mathsf{gen}\,h\,t\,c) = h\,c \\
\mathsf{Stream}\beta_2 & : \mathsf{tail}\,(\mathsf{gen}\,h\,t\,c) = \mathsf{gen}\,h\,t\,(t\,c)
\end{array}
$$

Streams have two destructors: head and tail. The tail can be applied any number of times to the stream, so indeed this gives us infinite number of $\mathsf{Tm}\,A$s from a $u : \mathsf{Tm}\,(\mathsf{Stream}\,A)$:

$$\mathsf{head}\,u,\quad \mathsf{head}\,(\mathsf{tail}\,u),\quad \mathsf{head}\,(\mathsf{tail}\,(\mathsf{tail}\,u)),\quad \mathsf{head}\,(\mathsf{tail}\,(\mathsf{tail}\,(\mathsf{tail}\,u))),\quad \ldots$$

Streams can be made using the generator (coiterator, consructor, corecursor): the input of the generator is a type of seeds (C), functions h and t and a seed (an element of $\mathsf{Tm}\,C$) from which the stream grows out. The h tells us how to compute the current head element from the seed, while t says how to advance the seed (make a step): this is what the β rules express.

The stream containing the natural numbers $0, 1, 2, 3, \ldots$ is given by the following term where $C := \mathsf{Nat}$:

$$
\begin{array}{l}
\mathsf{nats} : \mathsf{Tm}\,(\mathsf{Stream}\,\mathsf{Nat}) \\
\mathsf{nats} := \mathsf{gen}\,(\lambda n.n)\,\mathsf{suc}\,\mathsf{zero}
\end{array}
$$

Indeed,

$$
\begin{aligned}
&\mathsf{head}\ \mathsf{nats} && = \\
&\mathsf{head}\ (\mathsf{gen}\ (\lambda n.n)\ \mathsf{suc}\ \mathsf{zero}) && =(\mathsf{Stream}\beta_1) \\
&(\lambda n.n)\ \mathsf{zero} && = \\
&\mathsf{zero}, && \\
&\mathsf{head}\ (\mathsf{tail}\ \mathsf{nats}) && = \\
&\mathsf{head}\ (\mathsf{tail}\ (\mathsf{gen}\ (\lambda n.n)\ \mathsf{suc}\ \mathsf{zero})) && =(\mathsf{Stream}\beta_2) \\
&\mathsf{head}\ (\mathsf{gen}\ (\lambda n.n)\ \mathsf{suc}\ (\mathsf{suc}\ \mathsf{zero})) && =(\mathsf{Stream}\beta_1) \\
&(\lambda n.n)\ (\mathsf{suc}\ \mathsf{zero}) && = \\
&\mathsf{suc}\ \mathsf{zero}, && \\
&\mathsf{head}\ (\mathsf{tail}\ (\mathsf{tail}\ \mathsf{nats})) && = \\
&\mathsf{head}\ (\mathsf{tail}\ (\mathsf{tail}\ (\mathsf{gen}\ (\lambda n.n)\ \mathsf{suc}\ \mathsf{zero}))) && =(\mathsf{Stream}\beta_2) \\
&\mathsf{head}\ (\mathsf{tail}\ (\mathsf{gen}\ (\lambda n.n)\ \mathsf{suc}\ (\mathsf{suc}\ \mathsf{zero}))) && =(\mathsf{Stream}\beta_2) \\
&\mathsf{head}\ (\mathsf{gen}\ (\lambda n.n)\ \mathsf{suc}\ (\mathsf{suc}\ (\mathsf{suc}\ \mathsf{zero}))) && =(\mathsf{Stream}\beta_1) \\
&(\lambda n.n)\ (\mathsf{suc}\ (\mathsf{suc}\ \mathsf{zero})) && = \\
&\mathsf{suc}\ (\mathsf{suc}\ \mathsf{zero}), &&
\end{aligned}
$$

and so on.

Exercise 139. *Define pointwise addition on two* Stream Nat*s, and define the* map *function on streams.*

We cannot define the filter function on streams because it is not necessarily productive.

Every strictly positive type operator determines a coinductive type. In the following, we extend the language of inductive type with coinductive types (actually we don't need inductive types, only SPos and map).

Ötlet: kezeljük párban a streameket, ekkor a head és a tail is tud a pár megfelelő elemével dolgozni.

addStreams : Tm(Stream Nat ⇒ Stream Nat ⇒ Stream Nat)

addStreams := lam λa. lam λb. gen

(lam λp. add · (head (fst p)) · (head (snd p))) ← head függvény

(lam λp. (tail (fst p), tail (snd p))) ← tail függvény

(a, b) ← kiinduló seed

Ötlet: legyen a Stream A seed típusa, a seed az aktuális streak, a head alkalmazza az f függvényt rá, a tail pedig eggyel lépteti.

mapStream : (Tm A → Tm B) → Tm (Stream A ⇒ Stream B)

mapStream f := lam λs. gen

(lam λx. f · (head x)) ← head függvény: f alkalmazása az aktuális headre

(lam λx. tail x) ← tail függvény: a stream továbbléptetése

s ← kiinduló seed

Definition 140 (Language with inductive and coinductive types)**.** *A second-order model extends Definition 127 with the following components.*

$$
\begin{aligned}
&\mathsf{Coind} &&: \mathsf{SPos}\,\mathsf{F} \to \mathsf{Ty}\\
&\mathsf{des} &&: (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \mathsf{Tm}\,(\mathsf{Coind}\,F) \to \mathsf{Tm}\,\big(F\,(\mathsf{Coind}\,F)\big)\\
&\mathsf{gen} &&: (F : \mathsf{Ty} \to \mathsf{Ty}) \to \{\mathsf{SPos}\,\mathsf{F}\} \to \big(\mathsf{Tm}\,A \to \mathsf{Tm}\,(F\,A)\big) \to \mathsf{Tm}\,A \to \mathsf{Tm}\,(\mathsf{Coind}\,F)\\
&\mathsf{Coind}\beta &&: \mathsf{des}\,F\,(\mathsf{gen}\,F\,f\,a) = \mathsf{map}\,F\,(\mathsf{gen}\,F\,f)\,(f\,a)\\
&\mathsf{Coind}\eta &&: (z : \mathsf{Tm}\,A \to \mathsf{Tm}\,(\mathsf{Coind}\,F)) \to \Big(\big(\lambda z.\mathsf{des}\,F\,(z\,a)\big) = \big(\lambda z.\mathsf{map}\,F\,z\,(f\,a)\big)\Big) \to z = \mathsf{gen}\,F\,f
\end{aligned}
$$

Exercise 141. *Show that the destructors and the generator of streams can be given using* $\mathsf{Stream}\,A := \mathsf{Coind}\,(\lambda X.\,A \times X)$.

Exercise 142. *Define addition and multiplication on conatural numbers (*$\mathsf{Coind}\,(\lambda X.Idotp\top + X)$*)!.*

Exercise 143. *Which inductive type is the dual of streams?*

Exercise 144. *What is the dual of lists, that is,* $\mathsf{Coind}\,(\lambda X.\,\top + A \times X)$*?*

Exercise 145. *Define the coinductive type of machines which can receive a natural number, they can print out a number and there is a button on them. Using the generator, define an summation machine, which adds all the input numbers, returns the sum when asked, and the button resets the number to 0. Define another machine in the same type which prints the maximum of all the input numbers.*

Exercise 146. *A* $\mathsf{Coind}\,F$ *típus az* F *operátor fixpontja: mutassuk meg, hogy* $F\,(\mathsf{Coind}\,F) \cong \mathsf{Coind}\,F$.

Exercise 147. *Show* $\mathsf{Coind}\,(\lambda X.\,A) \cong A$ *for every* A.

Food for thought 148. *The dual of iterator is generator. What is the dual of the recursor?*

The Flip Side of Inductive Types

In Section 3.5, inductive types were described by their **constructors** — you build a tree from the bottom up, and it always has finite depth. You can pick it up and look at the whole thing.

Coinductive types are the **exact opposite**. They are described by their **destructors** — you don't build the whole thing; instead you describe how to *observe* one layer of it at a time. Because you only ever look at a finite piece, the structure can be **infinitely deep** without causing a problem.

Streams: The Most Famous Coinductive Type

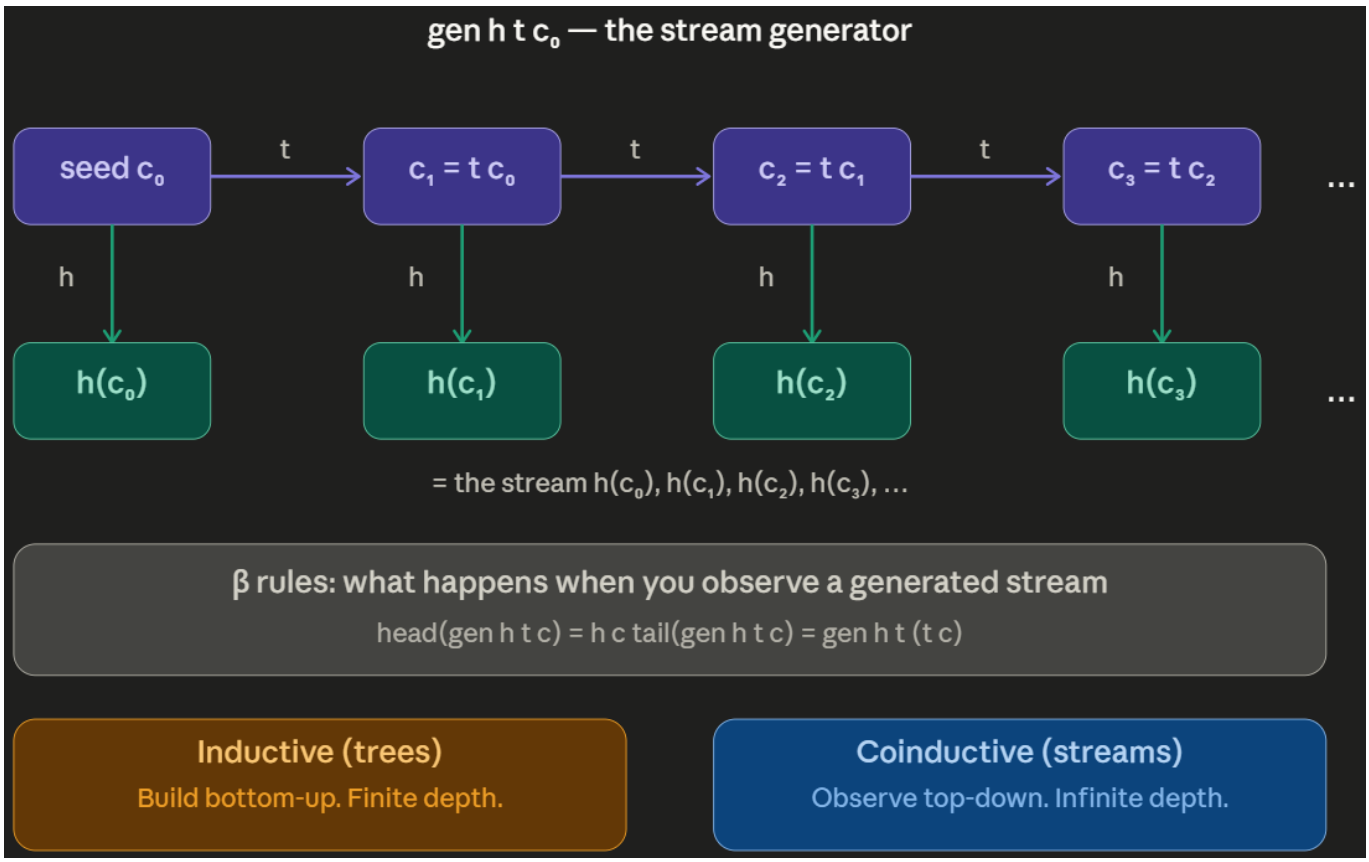
A Stream A is an infinite list of A-values. You never finish building it — but you can always look at:

- head s — the current element
- tail s — the rest of the stream

To *create* a stream, you use the **generator** gen. You don't specify all infinitely many elements. Instead, you provide:

1. A **seed** c — your starting state (like a counter set to 0)
2. A **head function** h — given the current state, produce the current element
3. A **tail function** t — given the current state, advance to the next state

The stream then produces elements lazily, on demand:



Why "infinite" doesn't cause a problem

The generator is always **productive**: every time you apply tail, you get something. You never get "stuck." This mirrors why inductive types are safe: the iterator always **terminates**. Both guarantee that nothing loops forever.

Think of it like a server: it runs forever, but at any given moment you only ask it one question at a time (one head or tail). You never have to wait for it to "finish."

The η rule (bisimulation) says: two streams are equal if they agree on *every* observation — every head after every sequence of tails. You can't tell them apart, so they *are* the same. This is why it's called bisimulation.

The general pattern (Definition 140) extends this to *any* SPos functor F:

- CoInd F is the **greatest fixpoint** of F (dual to Ind F as the least fixpoint)
- des peels off one layer: des : Tm(CoInd F) → Tm(F(CoInd F))
- gen builds from a seed: given f : A → F A, gen f : A → CoInd F

3.7.2 System F

In the next language, ∀ can appear anywhere, not only at the front of a type. It was discovered independently by Jean-Yves Girard [Gir72] and John C. Reynolds [Rey74], the former called it System F, the latter called it polymorphic lambda calculus.

Definition 152 (System F)**.** *A second-order model contains the following components:*

$$\begin{array}{ll} \mathsf{Ty} & : \mathsf{Set} \\ - \Rightarrow - & : \mathsf{Ty} \to \mathsf{Ty} \to \mathsf{Ty} \\ \forall & : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Ty} \end{array} \qquad \begin{array}{ll} \mathsf{Tm} & : \mathsf{Ty} \to \mathsf{Set} \\ \mathsf{lam} & : (\mathsf{Tm}\, A \to \mathsf{Tm}\, B) \cong \mathsf{Tm}\,(A \Rightarrow B) : - \cdot - \\ \mathsf{Lam} & : ((X : \mathsf{Ty}) \to \mathsf{Tm}\,(A\, X)) \cong \mathsf{Tm}\,(\forall\, A) : - \bullet - \end{array}$$

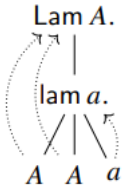
For the benefit of the reader, we also list the operations in concice derivation rule style (we don't write Ty and Tm anywhere):

$$\frac{A \qquad B}{A \Rightarrow B} \qquad \frac{x : A \vdash b : B}{\mathsf{lam}\, x.b : A \Rightarrow B} \qquad \frac{f : A \Rightarrow B \qquad a : A}{f \cdot a : B} \qquad \frac{}{(\mathsf{lam}\, x.b) \cdot a = b[x \mapsto a]} \qquad \frac{f : A \Rightarrow B}{f = \mathsf{lam}\, x.f \cdot x}$$

$$\frac{X \vdash A}{\forall X.A} \qquad \frac{X \vdash a : A}{\mathsf{Lam}\, X.a : \forall X.A} \qquad \frac{f : \forall X.A \qquad C}{f \bullet C : A[X \mapsto C]} \qquad \frac{}{(\mathsf{Lam}\, X.a) \bullet C = a[X \mapsto C]} \qquad \frac{f : \forall X.A}{f = \mathsf{Lam}\, X.f \bullet X}$$

Derivation of the polymorphic identity function (we made the implicit arguments of lam explicit):

$$\frac{\dfrac{\dfrac{}{A, a : A \vdash a : A}}{A \vdash \mathsf{lam}\,\{A\}\,\{A\}\,(a.a) : A \Rightarrow A}}{\mathsf{Lam}\, A.\, \mathsf{lam}\,\{A\}\,\{A\}\,(a.a) : \forall A.\, A \Rightarrow A}$$



Exercise 153. *Define the following programs (derive them in any second-order model).*

$$\begin{array}{ll} \mathsf{const} & : \mathsf{Tm}\,(\forall \lambda A.\, \forall \lambda B.\, A \Rightarrow B \Rightarrow A) \\ \mathsf{comp} & : \mathsf{Tm}\,(\forall \lambda A.\, \forall \lambda B.\, \forall \lambda C.\, (B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ \mathsf{comp}' & : \mathsf{Tm}\,(\forall \lambda B.\, \forall \lambda C.\, (B \Rightarrow C) \Rightarrow \forall \lambda A.\, (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ \mathsf{S} & : \mathsf{Tm}\,(\forall \lambda A.\, \forall \lambda B.\, \forall \lambda C.\, (A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C) \\ \mathsf{flip} & : \mathsf{Tm}\,(\forall \lambda A.\, \forall \lambda B.\, \forall \lambda C.\, (A \Rightarrow B \Rightarrow C) \Rightarrow B \Rightarrow A \Rightarrow C) \end{array}$$

const := Lam λA. Lam λB. lam λa. lam λb. a

comp := Lam λA. Lam λB. Lam λC. lam λf. lam λg. lam λx. g · (f · x)

comp' := Lam λB. Lam λC. lam λf. Lam λA. lam λg. lam λx. g · (f · x)

S := Lam λA. Lam λB. Lam λC. lam λf. lam λg. lam λx. f · x · (g · x)

flip := Lam λA. Lam λB. Lam λC. lam λf. lam λy. lam λx. f · x · y

Exercise 154. *Show that any second-order model of System F gives rise to a second-order model of Hindley-Milner.*

Remark 155. *To rule out trivial solutions to the previous exercise, redo it inside the theory of signatures (see Section 3.12): define a morphism of from the signature of System F to the signature of Hindley–Miler; this also gives rise to a functor between the categories of models.*

All inductive and coiductive types described in Section 3.5, 3.6 are already supported by System F (with their β rules but without η rules). This method is called *Church encoding*, it is a generalisation of how Alonzo Church encoded natural numbers in the untyped lambda calculus (see the next section). The two-element type is defined as follows.

$$\begin{aligned}
&\mathsf{Bool} && := \forall \lambda A.\, A \Rightarrow A \Rightarrow A\\
&\mathsf{true} && := \mathsf{Lam}\, \lambda A.\, \mathsf{lam}\, \lambda t.\, \mathsf{lam}\, \lambda f.\, t\\
&\mathsf{false} && := \mathsf{Lam}\, \lambda A.\, \mathsf{lam}\, \lambda t.\, \mathsf{lam}\, \lambda f.\, f\\
&\mathsf{ite} && : \mathsf{Tm}\, \mathsf{Bool} \to \mathsf{Tm}\, (\forall \lambda A.\, A \Rightarrow A \Rightarrow A)\\
&\mathsf{ite} && := \lambda b.\, b\\
&\mathsf{Bool}\beta_1 && : \mathsf{ite}\, \mathsf{true} \bullet A \cdot t \cdot f = (\lambda b.\, b)\, \mathsf{true} \bullet A \cdot t \cdot f = \mathsf{true} \bullet A \cdot t \cdot f = (\mathsf{Lam}\, \lambda A.\, \mathsf{lam}\, \lambda t.\, \mathsf{lam}\, \lambda f.\, t) \bullet A \cdot t \cdot f = t\\
&\mathsf{Bool}\beta_2 && : \mathsf{ite}\, \mathsf{false} \bullet A \cdot t \cdot f = (\lambda b.\, b)\, \mathsf{false} \bullet A \cdot t \cdot f = \mathsf{false} \bullet A \cdot t \cdot f = (\mathsf{Lam}\, \lambda A.\, \mathsf{lam}\, \lambda t.\, \mathsf{lam}\, \lambda f.\, f) \bullet A \cdot t \cdot f = f
\end{aligned}$$

The intuition is that a term in the above Bool type can only do two things: return its first or the second argument. In particular, it cannot look into the type A, it has to work in a uniform (parametric [Rey83, Wad89], natural [Awo10]) way for all As.

Note that Bool cannot be encoded in Hindley–Milner.

Exercise 156. *We defined the type of itertor above so that it can be easily implemented. Show that its type is isomorphic to the type of the usual if-then-else operator:*

$$(\mathsf{Tm}\, \mathsf{Bool} \to \mathsf{Tm}\, (\forall \lambda A.A \Rightarrow A \Rightarrow A)) \cong (\{A : \mathsf{Ty}\} \to \mathsf{Tm}\, \mathsf{Bool} \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A \to \mathsf{Tm}\, A).$$

Church encoding of a few more types:

empty	$\bot$	$\forall \lambda C.C$
unit	$\top$	$\forall \lambda C.C \Rightarrow C$
three-element		$\forall \lambda C.C \Rightarrow C \Rightarrow C \Rightarrow C$
sum	$A + B$	$\forall \lambda C.(A \Rightarrow C) \Rightarrow (B \Rightarrow C) \Rightarrow C$
product	$A \times B$	$\forall \lambda C.(A \Rightarrow B \Rightarrow C) \Rightarrow C$
natural numbers	Nat	$\forall \lambda C.C \Rightarrow (C \Rightarrow C) \Rightarrow C$
list of As	List A	$\forall \lambda C.C \Rightarrow (A \Rightarrow C \Rightarrow C) \Rightarrow C$
binary trees	Tree	$\forall \lambda C.C \Rightarrow (C \Rightarrow C \Rightarrow C) \Rightarrow C$

Exercise 157. *Derive the constructors and iterators for the above types, show that their computation rules hold.*

⊥ := ∀λC. C

konstruktora:

nincs – pont ez a lényege, így nincs szükség β szabályra, hiszen nincs mit redukálni

eliminátora:

exfalso : Tm ⊥ → Tm A

exfalso t := t • A

⊤ := ∀λC. C ⇒ C

konstruktora:

tt := Lam λC. lam λc. c (minden C-re ugyanazt adja, tehát teljesül)

iterátora:

nincs, csak egy elem van

Three := ∀λC. C ⇒ C ⇒ C ⇒ C

konstruktorai:
zero := Lam λC. lam λx. lam λy. lam λz. x
one := Lam λC. lam λx. lam λy. lam λz. y
two := Lam λC. lam λx. lam λy. lam λz. z

iterátora:
ite : Tm C → Tm C → Tm C → Tm Three → Tm C
ite a b c w := w • C · a · b · c

levezetési szabályok:
ite zero one two zero = zero • C · a · b · c = (∀β)
(lam λx. lam λy. lam λz. x) · a · b · c = (⇒β)
(lam λy. lam λz. a) · b · c = (⇒β)
(lam λz. a) · c = (⇒β)
a

A másik kettő hasonló.

A + B := ∀λC. (A ⇒ C) ⇒ (B ⇒ C) ⇒ C

konstruktorai:
inl a := Lam λC. lam λf. lam λg. f · a
inr b := Lam λC. lam λf. lam λg. g · b

elágazása:
case : (Tm A → Tm C) → (Tm B → Tm C) → Tm (A + B) → Tm C
case f g w := w • C · (lam λa. f a) · (lam λb. g b)

levezetési szabályok:
case f g (inl a) = (definíció)
(Lam λC. lam λl. lam λr. l · a) • C · (lam λa. f a) · (lam λb. g b) = (∀β)
(lam λl. lam λr. l · a) · (lam λa. f a) · (lam λb. g b) = (⇒β)
(lam λr. (lam λa. f a) · a) · (lam λb. g b) = (⇒β)
(lam λa. f a) · a = (⇒β)
f a

A másik oldal hasonló, g b-nek kell kijönnie.

A × B := ∀λC. (A ⇒ B ⇒ C) ⇒ C

konstruktorai:
pair a b := Lam λC. lam λf. f · a · b

kicsomagolása:
fst w := w • A · (lam λa. lam λ_. a)
snd w := w • B · (lam λ_. lam λb. b)

levezetési szabályok:
fst (pair a b) = (definíció)
(Lam λC. lam λf. f · a · b) • A · (lam λx. lam λ_. x) = (∀β)
(lam λf. f · a · b) · (lam λx. lam λ_. x) = (⇒β)
(lam λx. lam λ_. x) · a · b = (⇒β)
(lam λ_. a) · b = (⇒β)
a

A másik is hasonlóan teljesül, b jön ki 😊

Nat := ∀λC. C ⇒ (C ⇒ C) ⇒ C

konstruktorai:
zero := Lam λC. lam λz. lam λs. z
suc n := Lam λC. lam λz. lam λs. s · (n • C · z · s)

iterátora:
ite : Tm A → (Tm A → Tm A) → Tm Nat → Tm A
ite z s n := n • A · z · (lam s)

Food for thought 158. *Show that any inductive type encoded by a strictly positive functor can be derived in System F (without η rule).*

We can encode the other quantifier:

$$\begin{aligned}&\exists : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Ty}\\ &\exists F := \forall\, \lambda C.(\forall\, \lambda X.F\, X \Rightarrow C) \Rightarrow C\end{aligned}$$

Exercise 159. *Derive the constuctor, destructor and β rule:*

$$\begin{aligned}&\mathsf{mk}\exists : (X : \mathsf{Ty}) \to \mathsf{Tm}\,(F\, X) \to \mathsf{Tm}\,(\exists F)\\ &\mathsf{case} : ((X : \mathsf{Ty}) \to \mathsf{Tm}\,(F\, X) \to \mathsf{Tm}\, C) \to \mathsf{Tm}\,(\exists F) \to \mathsf{Tm}\, C\\ &\exists\beta \quad : \mathsf{case}\, w\, (\mathsf{mk}\exists\, X\, f) = w\, X\, f\end{aligned}$$

We exemplify the encoding of coinductive types through streams. A stream is given by a type C, a coalgebra on C and an element of C called the seed rfom which the stream grows out.

$$\begin{aligned}&\mathsf{Stream} : \mathsf{Ty} \to \mathsf{Ty}\\ &\mathsf{Stream}\, A := \exists\lambda C.\big((C \Rightarrow A) \times (C \Rightarrow C)\big) \times C\\ &\mathsf{head} : \mathsf{Tm}\,(\mathsf{Stream}\, A) \to \mathsf{Tm}\, A\\ &\mathsf{head} := \mathsf{case}\,\big(\lambda X\, w.\mathsf{fst}\,(\mathsf{fst}\, w) \cdot \mathsf{snd}\, w\big)\\ &\mathsf{tail} : \mathsf{Tm}\,(\mathsf{Stream}\, A) \to \mathsf{Tm}\,(\mathsf{Stream}\, A)\\ &\mathsf{tail} := \mathsf{case}\,\Big(\lambda X\, w.\mathsf{mk}\exists\, X\,\big(\mathsf{fst}\, w, \mathsf{snd}\,(\mathsf{fst}\, w) \cdot \mathsf{snd}\, w\big)\Big)\\ &\mathsf{gen} : \{C : \mathsf{Ty}\} \to (\mathsf{Tm}\, C \to \mathsf{Tm}\, A) \to (\mathsf{Tm}\, C \to \mathsf{Tm}\, C) \to \mathsf{Tm}\, C \to \mathsf{Tm}\,(\mathsf{Stream}\, A)\\ &\mathsf{gen}\,\{C\}\, h\, t\, c := \mathsf{mk}\exists\, C\,\big((\mathsf{lam}\, h, \mathsf{lam}\, t), c\big)\end{aligned}$$

Exercise 160. *Prove the computation rules of Church-encoded streams:*

$$\begin{aligned}&\mathsf{Stream}\beta_1 : \mathsf{head}\,(\mathsf{gen}\, h\, t\, c) = h\, c\\ &\mathsf{Stream}\beta_2 : \mathsf{tail}\,(\mathsf{gen}\, h\, t\, c) = \mathsf{gen}\, h\, t\,(t\, c)\end{aligned}$$

Food for thought 161. *Show that any coinductive type encoded by a strictly positive functor can be derived in System F (without η rule).*

Food for thought 162. *If we have internal parametricity [ACKS24, Lor25], the η rules also hold [Wad90]. Can we extend System F with internal parametricity without adding dependent types?*

Abstract types and interfaces can be encoded using existential types [MP88], and thus in System F. For example, if S is a type of Nat-stacks, then we want the following features:

empty	: Tm S	create an empty stack
push	: Tm (Nat ⇒ S ⇒ S)	put a Nat on the top of the stack
pop	: Tm (S ⇒ S)	remove an element from the top of the stack
top	: Tm (S ⇒ Nat)	look at what is in the top of the stack
isEmpty	: Tm (S ⇒ Bool)	let us know if the stack is empty

(Probably top should have type Tm (S ⇒ (Nat+⊤)), but that is not the point here.) A program which computes a Nat using an abstract stack interface has the following type:

$$(\exists \lambda S.\, S \times (\mathsf{Nat} \Rightarrow S \Rightarrow S) \times (S \Rightarrow S) \times (S \Rightarrow \mathsf{Nat}) \times (S \Rightarrow \mathsf{Bool})) \Rightarrow \mathsf{Nat}.$$

Exercise 163. *Define a term of the above type which creates a stack, then pushes 1, 2, 3 on it, pops one element, and returns what is on the top of the stack. Show that if the following equations hold for our stack implementation, the result will be 2:*

$$\mathsf{top} \cdot (\mathsf{pop} \cdot (\mathsf{push} \cdot n \cdot (\mathsf{push} \cdot m \cdot s))) = m.$$

This program behaves uniformly for any stack implementation. Of course, there can be trivial stack implementations which don't behave well. If we want to statically restrict our stack implementation to correct ones, we need dependent types (Section 3.9). Otherwise, we can just assume that the stack implementation behaves well.

Exercise 164. *How do you describe with equations such as the above that a stack implementation behaves correctly?*

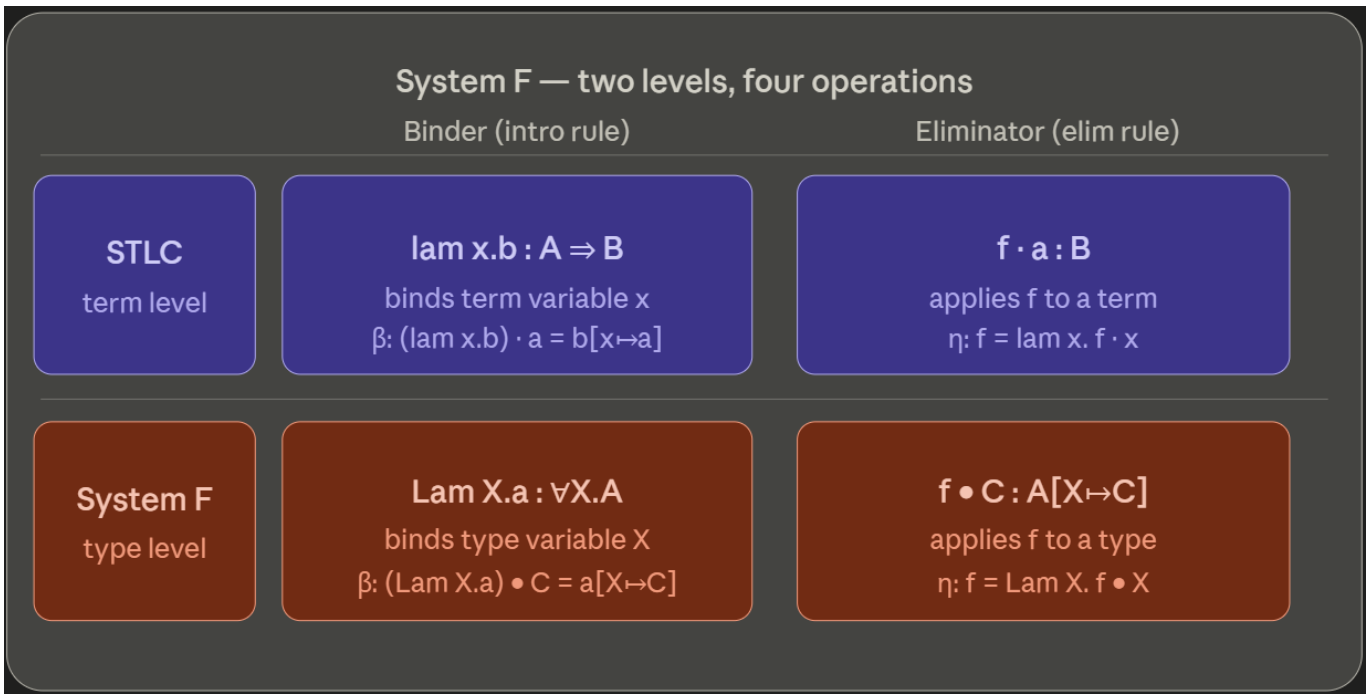
In STLC, every function is written for one specific pair of types. You write lam x. x for Nat ⇒ Nat, and if you want identity on Bool too, you write it again from scratch. That's tedious. System F solves this by adding a **second kind of abstraction and application** — one that operates at the *type* level rather than the *term* level.

Think of it like this: in STLC you can write a function that takes a *value* and returns a value (lam x. …). In System F, you can additionally write a function that takes a *type* and returns a term (Lam X. …). This lets you write one identity function that works for literally any type you hand it.

The key new pieces are ∀X. A (a type saying "for all types X, here is something of type A"), Lam X. a (a term that abstracts over a type variable X), and f • C (applying a polymorphic term to a concrete type C). The β rule says (Lam X. a) • C = a[X ↦ C] — substituting the type in, just as STLC's β rule substitutes a value.

What makes System F especially powerful is **Church encoding**: the ∀ is expressive enough to encode Bool, Nat, lists, trees, sums, products, and even existential types — all without adding any of them separately to the language. The trick is that a type like ∀C. C ⇒ (C ⇒ C) ⇒ C *already is* its own iterator: any term of that type, when applied to a base case and a step, runs the fold.

Here is the structure of the four operations at a glance:



3.8 PCF, Recursive types, Untyped lambda calculus

If we have System F-style polymorphism, we don't need to add (co)inductive types by hand into our language. There is another very common way to add features to our language: adding noise, or in other words, making the language less safe. On the other hand, these languages are Turing-complete.

S. 3.8.1 We can add a fixpoint-operator to our language and then we do not need iterators anymore, but we lose that every program terminates: we add meaningless recursive programs which simply loop. The language which has Nat without iterator but with a fixpoint operator is called PCF.

S. 3.8.2 We can add recursive types by saying that every type operator has a fixpoint: we lose the distinction between inductive and coinductive types and now not only terms, but types contain rubbish.

S. 3.8.3 We can remove all types, then we don't even need to add a fixpoint operator, we can just define it. This is the untyped lambda calculus. We have no guarantee that programs are meaningful, but we can implement anything.

3.8.1 PCF

A *fixpoint-operator* is given by $\mathsf{fix} : (\mathsf{Tm}\, A \to \mathsf{Tm}\, A) \to \mathsf{Tm}\, A$ which satisfies the equation $\mathsf{fix}\, f = f\, (\mathsf{fix}\, f)$.

Definition 170 (PCF). *A second-order model of PCF has two sorts (types and type-indexed terms) and the following additional components using concise derivation rule notation:*

$$\frac{A \qquad B}{A \Rightarrow B} \qquad \frac{x : A \vdash b : B}{\mathsf{lam}\, x.b : A \Rightarrow B} \qquad \frac{f : A \Rightarrow B \qquad a : A}{f \cdot a : B} \qquad \frac{}{(\mathsf{lam}\, x.b) \cdot a = b[x \mapsto a]} \qquad \frac{x : A \vdash t : A}{\mathsf{fix}\, x.t : A} \qquad \frac{}{\mathsf{fix}\, x.t = t[x \mapsto \mathsf{fix}\, x.t]}$$

$$\frac{}{\mathsf{Nat}} \qquad \frac{}{\mathsf{zero} : \mathsf{Nat}} \qquad \frac{n : \mathsf{Nat}}{\mathsf{suc}\, n : \mathsf{Nat}} \qquad \frac{b : \mathsf{Nat} \qquad t : A \qquad f : A}{\mathsf{ifZero}\, b\, t\, f : A} \qquad \frac{}{\mathsf{ifZero}\, \mathsf{zero}\, t\, f = t} \qquad \frac{}{\mathsf{ifZero}\, (\mathsf{suc}\, n)\, t\, f = f}$$

$$\frac{n : \mathsf{Nat}}{\mathsf{pred}\, n : \mathsf{Nat}} \qquad \frac{}{\mathsf{pred}\, (\mathsf{suc}\, n) = n} \qquad \frac{}{\mathsf{pred}\, \mathsf{zero} = \mathsf{zero}}$$

Example 171. *We want to define the $x \mapsto x * 2 + 1$ function in a way that it satisfies the following equation:*

$$f = \mathsf{lam}\, \lambda n.\, \mathsf{ifZero}\, n\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (f \cdot \mathsf{pred}\, n))).$$

This cannot be seen as a definition because f appears on the right hand side of the equation. We fix this by abstracting over the recursive call:

$$g : \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat}) \to \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat})$$
$$g\, f := \mathsf{lam}\, \lambda n.\, \mathsf{ifZero}\, n\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (f \cdot \mathsf{pred}\, n)))$$

The fixpoint of g satisfies the desired equation:

$$\begin{array}{ll}
\mathsf{fix}\, g : \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow \mathsf{Nat}) & \\
\mathsf{fix}\, g & =(\beta \textit{ rule for } \mathsf{fix}) \\
g\, (\mathsf{fix}\, g) & =(\textit{definition of } g) \\
\mathsf{lam}\, \lambda n.\, \mathsf{ifZero}\, n\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, n))) &
\end{array}$$

We also check that $\mathsf{fix}\, g$ *computes as expected:*

$$\begin{array}{ll}
\mathsf{fix}\, g \cdot \mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}) & =(\beta \textit{ rule for } \mathsf{fix}) \\
(\mathsf{lam}\, \lambda n.\, \mathsf{ifZero}\, n\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, n)))) \cdot \mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}) & =(\Rightarrow\beta) \\
\mathsf{ifZero}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}))\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero}))))) & =(\mathsf{ifZero}\textit{'s } \beta \textit{ rule}) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero})))) & =(\mathsf{pred}\textit{'s } \beta \textit{ rule}) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{suc}\, \mathsf{zero})) & =(\beta \textit{ rule for } \mathsf{fix}) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{lam}\, \lambda n.\, \mathsf{ifZero}\, n\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, n))) \cdot \mathsf{suc}\, \mathsf{zero})) & =(\Rightarrow\beta) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{ifZero}\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, (\mathsf{suc}\, \mathsf{zero}))))))) & =(\Rightarrow\beta) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, (\mathsf{suc}\, \mathsf{zero})))))) & =(\mathsf{pred}\textit{'s } \beta \textit{ rule}) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{zero})))) & =(\beta \textit{ rule for } \mathsf{fix}) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{lam}\, \lambda n.\, \mathsf{ifZero}\, n\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, n))) \cdot \mathsf{zero})))) & =(\Rightarrow\beta) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{ifZero}\, \mathsf{zero}\, (\mathsf{suc}\, \mathsf{zero})\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{fix}\, g \cdot \mathsf{pred}\, \mathsf{zero}))))))) & =(\mathsf{ifZero}\textit{'s } \beta \textit{ rule}) \\
\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, (\mathsf{suc}\, \mathsf{zero})))) &
\end{array}$$

Exercise 172. *Define the iterator of natural numbers and show its computation rules.*

ite : Tm A → (Tm A → Tm A) → Tm (Nat ⇒ A)

ite z s := fix λf. lam λn. ifZero n z (s · (f · pred n))

Exercise 173. *Show weak equational consistency for any second-order model: that is, if* $\mathsf{zero} = \mathsf{suc}\, \mathsf{zero}$*, then for all A and $u, v : \mathsf{Tm}\, A$, $u = v$.*

w : Tm (Nat ⇒ A)
w := lam λn. ifZero n u v

w · zero = (lam λn. ifZero n u v) · zero = ifZero zero u v = u (⇒β, majd ifZero β_1)
w · (suc zero) = (lam λn. ifZero n u v) · (suc zero) = ifZero (suc zero) u v = v (⇒β, majd ifZero β_2)

u = w · zero = w · (suc zero) = v

Exercise 174. *Show that using a second-order model of PCF, a second-order model of System T without η can be given. See also Remark 155.*

Exercise 175. *Which functions do the following PCF terms define?*

$$\mathsf{fix}\,(\lambda t.\,\mathsf{lam}\,\lambda x.\,x)$$
$$\mathsf{fix}\,(\lambda t.\,\mathsf{lam}\,\lambda x.\,\mathsf{ifZero}\,x\,x\,x)$$
$$\mathsf{fix}\,(\lambda t.\,\mathsf{lam}\,\lambda x.\,\mathsf{ifZero}\,x\,x\,(\mathsf{suc}\,\mathsf{zero}))$$
$$\mathsf{fix}\,(\lambda t.\,\mathsf{lam}\,\lambda x.\,\mathsf{ifZero}\,x\,(\mathsf{suc}\,\mathsf{zero})\,\mathsf{zero})$$
$$\mathsf{fix}\,\Big(\lambda t.\,\mathsf{lam}\,\lambda x.\,\mathsf{ifZero}\,x\,x\,\big(\mathsf{suc}\,(t\cdot\mathsf{pred}\,x)\big)\Big)$$
$$\mathsf{fix}\,\Big(\lambda t.\,\mathsf{lam}\,\lambda x.\,\mathsf{ifZero}\,x\,\mathsf{zero}\,\big(\mathsf{suc}\,(t\cdot\mathsf{pred}\,x)\big)\Big)$$
$$\mathsf{fix}\,\Big(\lambda t.\,\mathsf{lam}\,\lambda x.\,\mathsf{ifZero}\,x\,(\mathsf{suc}\,\mathsf{zero})\,\big(\mathsf{suc}\,(t\cdot\mathsf{pred}\,x)\big)\Big)$$

The price we had to pay for having all recursive functions is that we have non-productive terms: $\mathsf{fix}\,(\lambda x.\,x) : \mathsf{Tm}\,A$ for any A, and we have $\mathsf{fix}\,(\lambda x.\,x) = (\lambda x.\,x)(\mathsf{fix}\,(\lambda x.\,x)) = \mathsf{fix}\,(\lambda x.\,x)$, so we don't obtain any new information from computing. Similarly, $\mathsf{fix}\,\mathsf{suc} = \mathsf{suc}\,(\mathsf{fix}\,\mathsf{suc}) = \mathsf{suc}\,(\mathsf{suc}\,(\mathsf{fix}\,\mathsf{suc})) = \ldots$, which will be a very large term never terminating. Compare this with $\mathsf{fix}\,(\lambda_.\,t) = (\lambda_.\,t)\,(\mathsf{fix}\,(\lambda_.\,t)) = t$.

- identitásfüggvény
- identitásfüggvény
- ha x = 0 → 0, egyébként 1
- ha x = 0 → 1, egyébként 0
- rekurzióval ugyan, de az identitásfüggvényt adja
- hasonlóan rekurzióval az identitásfüggvényt adja
- lényegében suc (x-hez 1-et ad)

3.8.2 Recursive types

We have a fixpoint operator for $F : \mathsf{Ty} \to \mathsf{Ty}$ functions, that is, an operator $\mathsf{Fix} : (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Ty}$. There are two ways to express that $\mathsf{Fix}\,F$ is the fixpoint of F:

- *equirecursive types*, that is, we add the equation $\mathsf{Fix}\,F = F\,(\mathsf{Fix}\,F)$. This makes equality of types undecidable (and thus typechecking undecidable).
- *isorecursive types*: instead of adding an equation in types, we add an isomorphism $\mathsf{Fix}\,F \cong F\,(\mathsf{Fix}\,F)$.

We follow the latter route.

Definition 176 (A language with recursive types)**.** *A second-order model extends Fin (Definition 91) with the following components:*

$$\begin{aligned}
\mathsf{fix} &: (\mathsf{Tm}\,A \to \mathsf{Tm}\,A) \to \mathsf{Tm}\,A\\
\mathsf{fix}\beta &: \mathsf{fix}\,f = f\,(\mathsf{fix}\,f)\\
\mathsf{Fix} &: (\mathsf{Ty} \to \mathsf{Ty}) \to \mathsf{Ty}\\
\mathsf{con} &: \mathsf{Tm}\,(F\,(\mathsf{Fix}\,F)) \cong \mathsf{Tm}\,(\mathsf{Fix}\,F) : \mathsf{des}
\end{aligned}$$

Example 177 (Nat). *We define natural numbers as a recursive type. We only write the implicit argument* $\{\lambda X.\, \top + X\}$ *of* con *once.*

$$
\begin{array}{lll}
\mathsf{Nat} & := \mathsf{Fix}\, \lambda X.\, \top + X & \\
\mathsf{zero} & := \mathsf{con}\, \{\lambda X.\, \top + X\}\, (\mathsf{inl}\, \mathsf{tt}) & \\
\mathsf{suc} & : \mathsf{Tm}\, \mathsf{Nat} \to \mathsf{Tm}\, \mathsf{Nat} & \\
\mathsf{suc}\, n & := \mathsf{con}\, (\mathsf{inr}\, n) & \\
\mathsf{ite} & : \mathsf{Tm}\, A \to (\mathsf{Tm}\, A \to \mathsf{Tm}\, A) \to \mathsf{Tm}\, (\mathsf{Nat} \Rightarrow A) & \\
\mathsf{ite}\, z\, s & := \mathsf{fix}\, \lambda f.\, \mathsf{lam}\, \lambda n.\, \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (f \cdot m))\, (\mathsf{des}\, n) & \\
\mathsf{Nat}\beta_1 : & \mathsf{ite}\, z\, s \cdot \mathsf{zero} & =(\textit{definition}) \\
& (\mathsf{fix}\, \lambda f.\, \mathsf{lam}\, \lambda n.\, \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (f \cdot m))\, (\mathsf{des}\, n)) \cdot \mathsf{zero} & =(\mathsf{fix}\beta) \\
& (\mathsf{lam}\, \lambda n.\, \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{des}\, n)) \cdot \mathsf{zero} & =(\Rightarrow\beta) \\
& \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{des}\, \mathsf{zero}) & =(\textit{definition}) \\
& \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{des}\, (\mathsf{con}\, (\mathsf{inl}\, \mathsf{tt}))) & =(\mathsf{Fix}\beta) \\
& \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{inl}\, \mathsf{tt}) & =(+\beta_1) \\
& z & \\
\mathsf{Nat}\beta_2 : & \mathsf{ite}\, z\, s \cdot \mathsf{suc}\, n & =(\textit{definition}) \\
& (\mathsf{fix}\, \lambda f.\, \mathsf{lam}\, \lambda n.\, \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (f \cdot m))\, (\mathsf{des}\, n)) \cdot \mathsf{suc}\, n & =(\mathsf{fix}\beta) \\
& (\mathsf{lam}\, \lambda n.\, \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{des}\, n)) \cdot \mathsf{suc}\, n & =(\Rightarrow\beta) \\
& \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{des}\, (\mathsf{suc}\, n)) & =(\textit{definition}) \\
& \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{des}\, (\mathsf{con}\, (\mathsf{inr}\, n))) & =(\mathsf{Fix}\beta) \\
& \mathsf{case}\, (\lambda _.\, z)\, (\lambda m.\, s\, (\mathsf{ite}\, z\, s \cdot m))\, (\mathsf{inr}\, n) & =(+\beta_2) \\
& s\, (\mathsf{ite}\, z\, s \cdot n) &
\end{array}
$$

Food for thought 178. *Does* $\mathsf{Nat}\eta$ *hold?*

Example 179 (Stream). *We define streams as a recursive type.*

$$
\begin{array}{lll}
\mathsf{Stream} & : \mathsf{Ty} \to \mathsf{Ty} & \\
\mathsf{Stream}\, A & := \mathsf{Fix}\, \lambda A.\, A \times X & \\
\mathsf{head} & : \mathsf{Tm}\, (\mathsf{Stream}\, A) \to \mathsf{Tm}\, A & \\
\mathsf{head}\, t & := \mathsf{fst}\, (\mathsf{des}\, t) & \\
\mathsf{tail} & : \mathsf{Tm}\, (\mathsf{Stream}\, A) \to \mathsf{Tm}\, (\mathsf{Stream}\, A) & \\
\mathsf{tail}\, t & := \mathsf{snd}\, (\mathsf{des}\, t) & \\
\mathsf{gen} & : (\mathsf{Tm}\, C \to \mathsf{Tm}\, A) \to (\mathsf{Tm}\, C \to \mathsf{Tm}\, C) \to \mathsf{Tm}\, (C \Rightarrow \mathsf{Stream}\, A) & \\
\mathsf{gen}\, h\, t & := \mathsf{fix}\, \lambda f.\, \mathsf{lam}\, \lambda c.\, \mathsf{con}\, (h\, c, f \cdot t\, c) &
\end{array}
$$

Exercise 180 (long). *Prove the* β *rules for streams.*

Exercise 181 (long). *Show that for a stictly positive* $F : \mathsf{Ty} \to \mathsf{Ty}$, $\mathsf{Fix}\, F$ *satisfies all the rules of* $\mathsf{Ind}\, F$ *and* $\mathsf{Coind}\, F$.

We have meaningless types such as $D := \mathsf{Fix}\, (\lambda X.X)$ which gives as a $\mathsf{con} : \mathsf{Tm}\, D \to \mathsf{Tm}\, D$ and $\mathsf{des} : \mathsf{Tm}\, D \to \mathsf{Tm}\, D$, but we cannot create any element of D and cannot obtain any information from assumed elements of D.

3.8.3 Untyped lambda calculus

The simplest programming language with binders is the untyped lambda calculus.

Definition 182 (Untyped lambda calculus)**.**

$$\begin{aligned} \mathsf{Tm} &: \mathsf{Set} \\ - \cdot - &: \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} \\ \mathsf{lam} &: (\mathsf{Tm} \to \mathsf{Tm}) \to \mathsf{Tm} \\ \beta &: \mathsf{lam}\, f \cdot u = f\, u \end{aligned}$$

A fixpoint combinator:

$$\mathsf{Y} : \mathsf{Tm} := \mathsf{lam}\, \lambda f.(\mathsf{lam}\, \lambda x. f \cdot (x \cdot x)) \cdot (\mathsf{lam}\, \lambda x. f \cdot (x \cdot x)).$$

Exercise 183. *Show that* $\mathsf{Y} \cdot f = f \cdot (\mathsf{Y} \cdot f)$.

Legyen g := lam λx. f · (x · x). Ekkor Y · f = g · g.

Kibontva: g · g = (lam λx. f · (x · x)) · g = f · (g · g) = f · (Y · f)

Exercise 184. *Show that there are terms* zero, suc, ite : Tm *such that* $\mathsf{ite} \cdot z \cdot s \cdot \mathsf{zero} = z$ *and* $\mathsf{ite} \cdot z \cdot s \cdot (\mathsf{suc}\, t) = s \cdot (\mathsf{ite} \cdot z \cdot s \cdot t)$.

zero := lam λz. lam λs. z
suc := lam λn. lam λz. lam λs. s · (n · z · s)
ite := lam λz. lam λs. lam λn. n · z · s

Exercise 185. *Show that we have similar encoding for all inductive and coinductive types. What about* η *rules?*

We have another untyped language which has one more operator and one more equation, but is first-order:

We have another untyped language which has one more operator and one more equation, but is first-order:

Definition 186 (Untyped combinator calculus)**.**

$$\begin{aligned} \mathsf{Tm} &: \mathsf{Set} \\ - \cdot - &: \mathsf{Tm} \to \mathsf{Tm} \to \mathsf{Tm} \\ \mathsf{K} &: \mathsf{Tm} \\ \mathsf{S} &: \mathsf{Tm} \\ \mathsf{K}\beta &: \mathsf{K} \cdot a \cdot b = a \\ \mathsf{S}\beta &: \mathsf{S} \cdot f \cdot g \cdot a = f \cdot a \cdot (g \cdot a) \end{aligned}$$

See [AKSV23] for the relationship of untyped lambda and combinator calculi.

It is remarkable that Schönfinkel's untyped combinator calculus is still the simplest Turing complete language, even though it was the first such. Note that in contrast with STCC, untyped combinator calculus does not have normalisation.

TODO: add exercises from Hungarian course notes, including fixpoint operator for SK.

The Big Picture First

Everything so far in the notes (System T, STLC, Hindley–Milner, System F) was a **safe** language: every well-typed program is *guaranteed* to eventually finish. That safety came at a cost — there were things you simply could not write. Section 3.8 asks: what if we start removing those restrictions, one by one? Each step gives you more expressive power and takes away one safety guarantee.

3.8.1 — PCF: Adding fix for Terms

Imagine a recipe card that says: *"To make soup: heat some water, add ingredients, taste — if not ready, make soup again."* That recipe calls itself. In System T, this was forbidden — every program had to provably finish. In PCF, we add a single new operation called the **fixpoint operator** fix, which lets a term call itself. Now you can write "do this until I'm satisfied" — but the program might run forever and never be satisfied.

The formal rule is simply: fix f = f (fix f). Whatever f is, fix f is *the thing that stays the same when you apply f to it*. Think of it like two mirrors facing each other — fix f is the endless reflection.

What changes. System T's iterator for natural numbers could only do a bounded loop (exactly *n* steps, no more). PCF replaces it with ifZero, pred, and fix, which together let you write a loop that runs until some condition is met — but might never reach it. For example, you can now define the iterator yourself:

ite z s := fix λf. lam λn. ifZero n z (s · (f · pred n))

This says: *"to compute the iterator at n, check if n is zero; if so, return the base case; otherwise apply the step and call myself on n−1."* This is exactly what the safe iterator did, but now it's expressed through fix rather than built into the language as a primitive.

The trade-off. You gain the ability to write any recursive computation. You lose the guarantee that your program terminates. The price is the possibility of an infinite loop, like fix (λx. x) — which unfolds to itself forever and never produces a value.

3.8.2 — Recursive Types: Adding Fix for Types

Until now, types were always "made of smaller types." A list of numbers is built from numbers. A tree is built from smaller trees and leaves. The key word was *smaller* — you could always eventually reach the bottom. Now we add a **fixpoint operator for types**, written Fix F. This lets a type be made of… *itself*, without necessarily getting smaller.

Picture a Russian nesting doll — except when you open it, you find *the exact same doll* inside, not a slightly smaller one. Fix F is the type that satisfies Fix F ≅ F(Fix F): it IS its own unfolding.

What this gives you. You now have two operations on Fix F: con (fold in) and des (unfold out), and they are inverses of each other. This is exactly the isomorphism con : Tm(F(Fix F)) ⇌ Tm(Fix F) : des. What is remarkable is that this single type simultaneously behaves like an **inductive** type AND a **coinductive** type:

- As an inductive type: con is the constructor, and you build an iterator using the term-level fix to write a recursive fold over the structure.
- As a coinductive type: des is the destructor, and you build a generator using fix to write a corecursive unfold.

The notes show this concretely for streams: Stream A := Fix (λX. A × X). Opening the doll (des) reveals an A (the head) and another stream (the tail). Closing the doll (con) packages a head and a tail back into a stream.

The trade-off. The clean separation between *finite* (inductive) and *potentially infinite* (coinductive) structures disappears. A value of type Fix F could be either. With both fix (for terms) and Fix (for types), you can freely mix recursion and corecursion — and the language can genuinely loop forever.

3.8.3 — Untyped Lambda Calculus: Removing All Types

What if instead of carefully adding features, we just threw away ALL the rules? No types, no distinction between values and functions, no "this term has this type." Every term is just a *term*, and any term can be applied to any other term. You're left with a single rule: (lam λx. b) · a = b[x ↦ a]. That's the whole language.

This sounds chaotic — and it is — but something beautiful happens. Because you can now apply *anything* to *anything*, you can apply a term to **itself**: write x · x and nothing stops you. That self-application is the key. In typed languages, x · x was impossible: x would have to be simultaneously of type A and type A → B, which is a contradiction. In the untyped world, there is no contradiction because there are no types.

The Y combinator falls out for free. In PCF, we had to add fix as a primitive. In untyped λ-calculus, you can *derive* a fixpoint combinator using nothing but lambda and application:

Y := lam λf. (lam λx. f · (x · x)) · (lam λx. f · (x · x))

Let's see why Y · f = f · (Y · f). Let ω_f := lam λx. f · (x · x). Then Y · f = ω_f · ω_f. Unfolding: ω_f · ω_f = (lam λx. f · (x · x)) · ω_f = f · (ω_f · ω_f) = f · (Y · f). The self-application x · x is the engine — applying ω_f to itself causes it to feed itself back through f, creating the loop.

What you can and cannot encode. Because the Y combinator exists, you can define natural numbers, booleans, lists, and any inductive or coinductive type purely through Church-style encoding — exactly as in System F, but now without needing the polymorphic ∀. For example:

zero := lam λz. lam λs. z

suc := lam λn. lam λz. lam λs. s · (n · z · s)

ite := lam λz. lam λs. lam λn. n · z · s

And ite · z · s · zero = z, ite · z · s · (suc n) = s · (ite · z · s · n) — exactly the iterator rules, proved purely by β reduction.

The **η rules**, however, do not hold in general. In typed languages, η rules expressed *uniqueness* enforced by the type system ("the only term of this type satisfying these equations is this canonical form"). Without types, there is no mechanism to enforce uniqueness — you can always produce extra, weird terms that satisfy the β equations without being equal to the canonical form.

Putting It All Together

The three sections trace a single theme: **trading safety for power**. System T is maximally safe (total functions, strong normalization) but limited. Each extension relaxes one constraint — first allowing non-terminating computations (fix), then collapsing the inductive/coinductive distinction (Fix), then eliminating types entirely — until you reach the untyped λ-calculus, which is Turing complete but offers no guarantees whatsoever. The journey is deliberately one-way: every language in the chain contains the previous one, but you cannot get the safety properties back without adding new structure.